

Real-Time Recurrent Learning using Trace Units in Reinforcement Learning

Esraa Elelimy, Adam White*, Michael Bowling*, Martha White*

University of Alberta, Alberta Machine Intelligence Institute (Amii)

*Canada CIFAR AI Chair

elelimy, amw8, mbowling, whitem@ualberta.ca

Abstract

Recurrent Neural Networks (RNNs) are used to learn representations in partially observable environments. For agents that learn online and continually interact with the environment, it is desirable to train RNNs with real-time recurrent learning (RTRL); unfortunately, RTRL is prohibitively expensive for standard RNNs. A promising direction is to use linear recurrent architectures (LRUs), where dense recurrent weights are replaced with a complex-valued diagonal, making RTRL efficient. In this work, we build on these insights to provide a lightweight but effective approach for training RNNs in online RL. We introduce Recurrent Trace Units (RTUs), a small modification on LRUs that we nonetheless find to have significant performance benefits over LRUs when trained with RTRL. We find RTUs significantly outperform other recurrent architectures across several partially observable environments while using significantly less computation.

1 Introduction

Agents, animals, and people perceive their surrounding environment through imperfect sensory observations. When the state of the environment is partially observable, agents must construct and maintain their own state from the stream of observations. The constructed *agent state* summarizes past environment-agent interactions in a form that is useful to predict and control future interactions [40]. Recurrent Neural Networks (RNNs) provide a flexible architecture for constructing agent state [18, 20, 12, 5, 9].

While standard RNN architectures have been mainly supplanted by Transformers [43], in online reinforcement learning settings where the agent learns while interacting with the environment, RNNs remain a promising direction to pursue [16, 10]. There are two main issues that limit the use of self-attention mechanisms from Transformers in online learning. First, calculating the similarity between each pair of points results in a computational complexity that is a function of k^2 , where k is the sequence length. Moreover, calculating the similarity between all pairs ignores the temporal order of the data points, which limits the usefulness of self-attention when the data is temporally correlated [46]. Second, we need access to the whole sequence of observations before taking an action or updating the learnable parameters, which is impractical in continual learning. While recent works have reduced the complexity of transformers from quadratic in the sequence length to linear [14, 15, 36], the full sequence length is still needed to train such architectures. Gated Transformer-XL is an attempt to overcome this issue by keeping a moving window of previous observations [34]. A window of past observations does not scale well to long sequences—the computation is quadratic in the sequence length—and a window is one particular fixed function of history. The simpler recursive form in RNNs, on the other hand, can learn a great variety of functions of history from the data and is well suited for updating the state online for a sequential data stream and have been shown to outperform transformers in such settings [22].

A key open question is how to efficiently train RNNs in online RL. We can divide the literature into methods that approximate Real-Time Recurrent Learning (RTRL) and those that restrict the recurrent architecture. RTRL [45] exploits the recursive nature of the gradient for RNNs, carrying forward the needed gradient information instead of unrolling the recurrent dynamics back in time like Truncated Backpropagation Through Time (T-BPTT) [44]. RTRL avoids storing past data but is so computationally expensive as to be intractable for even moderately sized networks. Several methods approximate the RTRL gradient update, including NoBackTrack [32], Unbiased Online Recurrent Optimization [42, 4], Sparse N-Step Approximation (SnAp) [27]. All of these methods produce a biased gradient estimate. Other works have tried to approximate an unbiased gradient estimate of BPTT specifically for the case of policy gradient updates in RL. However, the approximation resulted in a high variance due to added stochasticity to the policy [1].

Methods in the second category usually restrict the RNN architecture to a diagonal RNN [8, 29], including Columnar Networks [17], the element-wise LSTM [16], and Independently Recurrent Neural Networks (IndRNNs) [19]. The RTRL algorithm is computationally efficient for such architectures. However, this approach sacrifices representation power and can perform poorly [17]. Recent work suggests overcoming the poor performance of diagonal RNNs with a small modification: having a complex-valued recurrent state instead of restricting it to real values [33]. In fact, as we will show in section 3.1, there exists an equivalence between using a dense linear recurrent layer and a diagonal recurrent layer with complex values, indicating no loss of representational capacity. LRUs have been combined with RTRL [47], though only empirically explored for supervised learning datasets.

In this work, we extend the insights from LRUs into the online RL setting. Our primary contribution are our experiments showing that such a lightweight recurrent architecture can outperform standard approaches like GRUs in RL, with significantly less computation. To obtain this result, we propose a small extension on LRUs, which we call Recurrent Trace Units (RTUs). RTUs incorporate nonlinearity into the recurrence and use a slightly different parameterization than LRUs, but one we find is more amenable to the use of RTRL in online RL than LRUs. We extend Proximal Policy Optimization (PPO) [39] to use RTRL, ablating the decision choices we propose. We provide an in-depth study in an animal-learning prediction benchmark, showing that RTUs scale better than GRUs with increasing compute and number of parameters and that RTUs outperform alternative diagonal recurrent architectures trained with RTRL. We then show across numerous control environments that RTUs have comparable or better performance, compared to GRUs and LRUs.

2 Background

We formalize the problem setting as a Partially Observable Markov Decision Process (POMDP). At each time step $t = 0, 1, 2, \dots$, the agent perceives an observation $\mathbf{x}_t$, a limited view of the state $\mathbf{s}_t \in \mathcal{S}$, and takes an action $A_t \in \mathcal{A}(\mathbf{s}_t)$. Depending on the action taken, the agent finds itself in a new state $\mathbf{s}_{t+1} \in \mathcal{S}$, observes the corresponding observation $\mathbf{x}_{t+1}$ and a reward $R_{t+1} \in \mathbb{R}$. In the online control setting, the agent’s goal is to maximize the sum of received rewards. It may also make predictions about its environment, such as future observations’ outcomes.

For prediction and control in a partially observable environment, the agent should use the stream of observations to construct its *agent state*. The agent state summarizes information from the history of the agent-environment interactions that are useful for prediction and control [40]. We could use the whole history up to t , namely $(\mathbf{x}_0, A_1, R_1, \mathbf{x}_1, A_2, R_2, \dots, \mathbf{x}_t)$, as the agent state. Though the history preserves all the information, it is not feasible to use directly. We want the agent to have constant memory and computation per time step and storing the whole history causes the memory and the computation to grow with time. Instead, the agent needs to compress this history into a concise representation. We refer to the agent’s internal representation of the history at time t as its agent state or its hidden state $\mathbf{h}_t$. The agent constructs its current agent state $\mathbf{h}_t \in \mathbb{R}^n$ from its previous agent state $\mathbf{h}_{t-1} \in \mathbb{R}^n$ and the recent observation $\mathbf{x}_t \in \mathbb{R}^d$ using a state-update function $\mathbf{g} : \mathbb{R}^n \times \mathbb{R}^d \rightarrow \mathbb{R}^n$: $\mathbf{h}_t = \mathbf{g}(\mathbf{h}_{t-1}, \mathbf{x}_t)$.

One way to learn this state-update function $\mathbf{g}$ is with a recurrent neural network (RNN). A simple form is a linear recurrent layer, where $\mathbf{g}(\mathbf{h}_{t-1}, \mathbf{x}_t) = \mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1}$ for weight matrices $\mathbf{W}_x$ and $\mathbf{W}_h$. We can also add a nonlinear activation, such as ReLU.

In general, we will write

$$\mathbf{h}_t = \mathbf{g}(\mathbf{h}_{t-1}, \mathbf{x}_t, \boldsymbol{\psi}),$$

where $\boldsymbol{\psi}$ are the learnable parameters in the network. The agent maps the agent state $\mathbf{h}_t$ to an output $\hat{y}_t$ and then receives a loss $\mathcal{L}_t \doteq \mathcal{L}(\hat{y}_t, y_t)$ indicating how far the output is from a target y_t . The agent updates $\boldsymbol{\psi}$ to minimize this loss over time.

Two main gradient-based algorithms are widely used to train RNNs: Truncated Backpropagation Through Time (TBPTT) and Real-Time Recurrent Learning (RTRL). T-BPTT specifies a truncation length T , which controls the number of steps considered when calculating the gradient [44]. As a result, the computation and memory complexities of T-BPTT are functions of the truncation length. Learning with T-BPTT involves a trade-off between the network’s ability to look further back in time and its compute and memory requirements. Picking a large T can be expensive, or require us to limit the network size, but picking too small of a T can cause the agent to miss important relationships and so result in poor performance.

Williams and Zipser (1989) introduced the Real-time Recurrent Learning algorithm (RTRL) as a learning algorithm for continual recurrent learning. Instead of unrolling the recurrent dynamics back in time, RTRL computes the gradient using the most recent observation, and the gradient is calculated and carried from the last step [45]. Assuming the network parameters have not changed, this recursive form gives the exact gradient and does not suffer from the truncation bias inherent to T-BPTT. We provide a more detailed background on the BPTT and RTRL in Appendix B. In reality, the agent updates its parameters frequently, so the gradient information saved from previous time steps is stale, i.e., calculated w.r.t old parameters; yet, under the assumption of small learning rates, RTRL is known to converge [45]. These properties make RTRL ideal for online learning, but unfortunately, there is a catch: it is quartic in the size of $\mathbf{h}_t$, which can be prohibitively expensive. For this reason, we pursue a restricted diagonal form in this work, for which RTRL is efficient and linear in $\mathbf{h}_t$.

3 Recurrent Trace Units

In this section, we introduce Recurrent Trace Units (RTUs). We start by revisiting why complex-valued diagonals represent dense recurrent layers, and why using real-valued diagonals is insufficient. We then introduce the specific form for RTUs that leverages this relationship. We then provide the RTRL update for RTUs, highlighting that it is simple to implement and linear in the hidden dimension. We finally contrast RTUs to LRUs and motivate why this small extension beyond LRUs is worthwhile.

3.1 Revisiting Complex-valued Diagonal Recurrence

Assume we have the recurrence relationship, with learnable parameters $\mathbf{W}_h \in \mathbb{R}^{n \times n}$ and $\mathbf{W}_x \in \mathbb{R}^{n \times d}$, $\mathbf{h}_t \doteq \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{u}(\mathbf{x}_t)$, where $\mathbf{u}$ can be any transformation of the inputs $\mathbf{x}_t$ before they are inputted into the recurrent layer. We can rewrite the square matrix $\mathbf{W}_h$ using an eigenvalue decomposition $\mathbf{W}_h = \mathbf{P} \boldsymbol{\Lambda} \mathbf{P}^{-1}$, where $\mathbf{P}$ contains the n linearly independent eigenvectors and $\boldsymbol{\Lambda} \in \mathbb{C}^{n \times n}$ is a diagonal matrix with the corresponding eigenvalues. Then we have that

$$\mathbf{h}_t = \mathbf{P}(\boldsymbol{\Lambda} \mathbf{P}^{-1} \mathbf{h}_{t-1} + \mathbf{P}^{-1} \mathbf{W}_x \mathbf{u}(\mathbf{x}_t)) \implies \mathbf{P}^{-1} \mathbf{h}_t = \boldsymbol{\Lambda} \mathbf{P}^{-1} \mathbf{h}_{t-1} + \mathbf{P}^{-1} \mathbf{W}_x \mathbf{u}(\mathbf{x}_t)$$

By defining $\bar{\mathbf{h}}_t \doteq \mathbf{P}^{-1} \mathbf{h}_t \in \mathbb{C}^n$ and $\bar{\mathbf{W}}_x \doteq \mathbf{P}^{-1} \mathbf{W}_x \in \mathbb{C}^{n \times d}$, we get a new recurrence $\bar{\mathbf{h}}_t = \boldsymbol{\Lambda} \bar{\mathbf{h}}_{t-1} + \bar{\mathbf{W}}_x \mathbf{u}(\mathbf{x}_t)$.

We can see $\bar{\mathbf{h}}_t$ and $\mathbf{h}_t$ are representationally equivalent: they are linearly weighted for downstream predictions, and so the linear transformation on $\bar{\mathbf{h}}_t$ can fold into this downstream linear weighting. But it is more computationally efficient to use $\bar{\mathbf{h}}_t$ with a diagonal weight matrix $\boldsymbol{\Lambda}$, meaning each hidden unit only has one recurrent relation instead of n . LRUs precisely leverage this equivalence [33]. Specifically, they learn a complex-valued $\bar{\mathbf{h}}_t$, and use $\text{Re}(\bar{\mathbf{W}} \bar{\mathbf{h}}_t)$ as an input to an MLP for downstream nonlinearity.

Since we did not impose constraints on the matrix $\mathbf{W}_h$, other than being diagonalizable, the eigenvalues of $\mathbf{W}_h$ can be complex or real numbers. Previous diagonal RNNs such as eLSTM [16], Columnar networks [17], and IndRNN [19] use only real-valued diagonal matrices. Having only real-valued diagonals assumes that the matrix $\mathbf{W}_h$ is symmetric. We provide a small experiment in Appendix A.3

showing that this assumption does not hold even in the simplest setting and that complex eigenvalues do arise. We also investigate whether this result can be extended beyond linear recurrence, and largely obtain a negative theoretical result (see Appendix A.1 and A.2).

3.2 The RTU Parameterization

A complex number can be represented in three ways: $a + bi$ (the real representation), $r \exp(i\theta)$ (the exponential representation), and $r(\cos(\theta) + i \sin(\theta))$ (the cosine representation). Mathematically, these three representations are equivalent, but do they affect learning differently? Orvieto et al. [33] empirically showed that using the exponential representation resulted in a better-behaved loss function than the real representation on a simple task; we provide some discussion in Appendix D.1 further motivating why the real representation is less stable. We chose instead to pursue the cosine representation, because it allows us to represent the complex hidden vector as two real-valued vectors. The remainder of this section outlines RTUs, with and without nonlinearity in the recurrence.

Our goal is to learn a complex-valued diagonal matrix with weights $\lambda_k = r_k(\cos(\theta_k) + i \sin(\theta_k))$ on the diagonal, for $k = 1, \dots, n$. Multiplying by a complex number is equivalent to multiplying by a 2×2 block matrix with a rescaling. We can use this rotational form to avoid explicitly using complex numbers, and instead use two real-values for each complex-valued hidden node. We write this real-valued matrix $\mathbf{\Lambda} \in \mathbb{R}^{2n \times 2n}$ as blocks of rotation matrices¹

$$\mathbf{\Lambda} = \begin{bmatrix} \mathbf{c}_1 & & \\ & \cdots & \\ & & \mathbf{c}_n \end{bmatrix} \quad \text{where} \quad \mathbf{c}_k = \mathbf{r}_k \begin{bmatrix} \cos(\theta_k) & -\sin(\theta_k) \\ \sin(\theta_k) & \cos(\theta_k) \end{bmatrix}. \quad (1)$$

Each element of $\mathbf{h}_t = \mathbf{\Lambda} \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t \in \mathbb{R}^{2n}$ has two components $\mathbf{h}_t^{c_1}, \mathbf{h}_t^{c_2}$, updated recursively

$$\begin{aligned} \mathbf{h}_t^{c_1} &= \mathbf{r} \cos(\boldsymbol{\theta}) \odot \mathbf{h}_{t-1}^{c_1} - \mathbf{r} \sin(\boldsymbol{\theta}) \odot \mathbf{h}_{t-1}^{c_2} + \mathbf{W}_x^{c_1} \mathbf{x}_t, \\ \mathbf{h}_t^{c_2} &= \mathbf{r} \cos(\boldsymbol{\theta}) \odot \mathbf{h}_{t-1}^{c_2} + \mathbf{r} \sin(\boldsymbol{\theta}) \odot \mathbf{h}_{t-1}^{c_1} + \mathbf{W}_x^{c_2} \mathbf{x}_t. \end{aligned}$$

We finally combine the new recurrent states into one state $\mathbf{h}_t \doteq [\mathbf{f}(\mathbf{h}_t^{c_1}); \mathbf{f}(\mathbf{h}_t^{c_2})]$, potentially using a non-linearity $\mathbf{f}$ after the recurrence.

We also adopt two parameterization choices made in LRUs that showed improved performance. The first is learning logarithmic representations of the parameters rather than learning them directly: instead of learning $\mathbf{r}$ and $\boldsymbol{\theta}$, the network learns $\boldsymbol{\nu}^{\log}$ and $\boldsymbol{\theta}^{\log}$, where $\mathbf{r} \doteq \exp(-\boldsymbol{\nu})$, $\boldsymbol{\nu} = \exp(\boldsymbol{\nu}^{\log})$, and $\boldsymbol{\theta}^{\log} \doteq \log(\boldsymbol{\theta})$. This re-parameterization restricts the $\mathbf{r}$ to be $\in (0, 1]$, required for stability. We found these modifications to improve stability of RTUs (see Appendix C). The second parameterization choice we adopt from LRUs is to multiply by the input $(\mathbf{W}_x \mathbf{x}_t)_k$ with a normalization factor of $\gamma_k = (1 - r_k^2)^{1/2}$. Putting this all together, the final formulation of RTUs is

$$\begin{aligned} \mathbf{h}_t^{c_1} &= \mathbf{g}(\boldsymbol{\nu}^{\log}, \boldsymbol{\theta}^{\log}) \odot \mathbf{h}_{t-1}^{c_1} - \boldsymbol{\phi}(\boldsymbol{\nu}^{\log}, \boldsymbol{\theta}^{\log}) \odot \mathbf{h}_{t-1}^{c_2} + \boldsymbol{\gamma} \odot \mathbf{W}_x^{c_1} \mathbf{x}_t, \\ \mathbf{h}_t^{c_2} &= \mathbf{g}(\boldsymbol{\nu}^{\log}, \boldsymbol{\theta}^{\log}) \odot \mathbf{h}_{t-1}^{c_2} + \boldsymbol{\phi}(\boldsymbol{\nu}^{\log}, \boldsymbol{\theta}^{\log}) \odot \mathbf{h}_{t-1}^{c_1} + \boldsymbol{\gamma} \odot \mathbf{W}_x^{c_2} \mathbf{x}_t, \\ \mathbf{h}_t &= [\mathbf{f}(\mathbf{h}_t^{c_1}); \mathbf{f}(\mathbf{h}_t^{c_2})]. \end{aligned} \quad (2)$$

where $\boldsymbol{\gamma} \in \mathbb{R}^n$ is the vector composed of $\gamma_k = (1 - \exp(-\exp(\nu_k^{\log}))^2)^{1/2}$ and

$$\begin{aligned} g(\nu_k, \theta_k) &\doteq \exp(-\exp(\nu_k^{\log})) \cos(\exp(\theta_k^{\log})), \\ \phi(\nu_k, \theta_k) &\doteq \exp(-\exp(\nu_k^{\log})) \sin(\exp(\theta_k^{\log})). \end{aligned} \quad (3)$$

Note that $\boldsymbol{\gamma}$ can be absorbed by $\mathbf{W}$, and so does not change representation capacity.

There are two ways to incorporate non-linearity into RTUs: inside the recurrence or after the recurrence. In the above, in Equation (2), the non-linearity is after the recurrence. These RTUs maintain the equivalence to a dense linear RNN, because the recurrence itself remains linear. We refer to this definition of RTUs as *Linear RTUs*, because the recurrence is linear, even though we have the ability to represent nonlinear functions by allowing for any nonlinear activation after the

¹We assume the matrix $\mathbf{\Lambda}$ has only complex eigenvalues, as the network can easily turn a complex eigenvalue into a real one by setting the imaginary component to 0.

recurrence. We also evaluated a different variation of RTUs where the non-linearity is added to the recurrence directly. These *Nonlinear RTUs* are written as

$$\begin{aligned} \mathbf{h}_t^{c1} &= \mathbf{f}(\mathbf{g}(\boldsymbol{\nu}^{\log}, \boldsymbol{\theta}^{\log}) \odot \mathbf{h}_{t-1}^{c1} - \phi(\boldsymbol{\nu}^{\log}, \boldsymbol{\theta}^{\log}) \odot \mathbf{h}_{t-1}^{c2} + \gamma \odot \mathbf{W}_x^{c1} \mathbf{x}_t) \\ \mathbf{h}_t^{c2} &= \mathbf{f}(\mathbf{g}(\boldsymbol{\nu}^{\log}, \boldsymbol{\theta}^{\log}) \odot \mathbf{h}_{t-1}^{c2} + \phi(\boldsymbol{\nu}^{\log}, \boldsymbol{\theta}^{\log}) \odot \mathbf{h}_{t-1}^{c1} + \gamma \odot \mathbf{W}_x^{c2} \mathbf{x}_t) \\ \mathbf{h}_t &= [\mathbf{h}_t^{c1}; \mathbf{h}_t^{c2}]. \end{aligned} \quad (4)$$

Notice now f —a nonlinear activation like ReLU—is used in the update to $\mathbf{h}_t^{c1}$ and $\mathbf{h}_t^{c2}$, and the final $\mathbf{h}_t$ simply stacks these two components. Nonlinear RTUs lose the equivalence to dense RNNs, though in our experiments, we find they perform as well or better than Linear RTUs.

3.3 The RTRL Update for RTUs

This section shows the RTRL updates for RTUs with more in-depth derivations in Appendix C. To keep notation simpler, we write the updates as if we are directly updating r and θ ; the updates for $\boldsymbol{\nu}^{\log}$ and $\boldsymbol{\theta}^{\log}$ are easily obtained then using the chain rule. The full derivation is in Appendix C.2.

Consider the partial derivative with respect to r_1 for the first RTU with input $\bar{x}_1 \doteq (\mathbf{W}_x^{c1} \mathbf{x}_t)_1$

$$h_{t,1}^{c1} = r_1 \cos(\theta_1) h_{t-1,1}^{c1} - r_1 \sin(\theta_1) h_{t-1,1}^{c2} + \sqrt{(1-r_1^2)} \bar{x}_1$$

Then

$$\frac{\partial \mathcal{L}_t}{\partial r_1} = \frac{\partial \mathcal{L}_t}{\partial h_{t,1}^{c1}} \frac{\partial h_{t,1}^{c1}}{\partial r_1} + \frac{\partial \mathcal{L}_t}{\partial h_{t,1}^{c2}} \frac{\partial h_{t,1}^{c2}}{\partial r_1}$$

because r_1 only impacts the two units in the first RTU, and derivatives w.r.t. the remaining hidden units are zero. Therefore, we just need to keep track of the vector of partial derivatives for these two components, $\mathbf{e}_t^{r,c1} \doteq [\frac{\partial h_{t,1}^{c1}}{\partial r_1}, \dots, \frac{\partial h_{t,n}^{c1}}{\partial r_n}]$ and $\mathbf{e}_t^{r,c2} \doteq [\frac{\partial h_{t,1}^{c2}}{\partial r_1}, \dots, \frac{\partial h_{t,n}^{c2}}{\partial r_n}]$ with recursive formulas

$$\begin{aligned} \mathbf{e}_t^{r,c1} &= \cos(\theta) \odot \mathbf{h}_{t-1}^{c1} + \mathbf{r} \odot \cos(\theta) \odot \mathbf{e}_{t-1}^{r,c1} - \sin(\theta) \odot \mathbf{h}_{t-1}^{c2} - \mathbf{r} \odot \sin(\theta) \odot \mathbf{e}_{t-1}^{r,c2} - \frac{\mathbf{r}}{\sqrt{1-r^2}} \odot \mathbf{W}_x^{c1} \mathbf{x}_t \\ \mathbf{e}_t^{r,c2} &= \cos(\theta) \odot \mathbf{h}_{t-1}^{c2} + \mathbf{r} \odot \cos(\theta) \odot \mathbf{e}_{t-1}^{r,c2} + \sin(\theta) \odot \mathbf{h}_{t-1}^{c1} + \mathbf{r} \odot \sin(\theta) \odot \mathbf{e}_{t-1}^{r,c1} - \frac{\mathbf{r}}{\sqrt{1-r^2}} \odot \mathbf{W}_x^{c2} \mathbf{x}_t \end{aligned}$$

We can similarly derive such traces for θ . The update to $\mathbf{r}$ involves first computing $\frac{\partial \mathcal{L}_t}{\partial h_{t,1}^{c1}}$, using backpropagation to compute gradients back from the output layer to the hidden layer; this step involves no gradients back-in-time. Then $\mathbf{r}$ is updated using the gradient $\frac{\partial \mathcal{L}_t}{\partial h_{t,1}^{c1}} \odot \mathbf{e}_t^{r,c1} + \frac{\partial \mathcal{L}_t}{\partial h_{t,1}^{c2}} \odot \mathbf{e}_t^{r,c2}$, which is linear in the size of $\mathbf{r} \in \mathbb{R}^n$, as the vectors $\mathbf{e}_t^{r,c1}, \mathbf{e}_t^{r,c2} \in \mathbb{R}^n$ can be updated with linear computation in the above recursion. This update is the RTRL update, with no approximation.

3.4 Contrasting to LRUs

RTUs are similar to LRUs, with two small differences. First, RTUs have real-valued hidden units, because the cosine representation is used instead of the exponential representation. Second, RTUs use nonlinear activations in the recurrence, making them no longer linear. Though again a minor difference, we find that incorporating nonlinearity in the recurrence can be beneficial. RTUs can be seen as a small generalization of LRUs, moving away from strict linearity—and thus motivating the name change—but nonetheless a generalization we find performs notably better in practice.

Let us now motivate the utility of moving to a cosine representation and real-valued traces. LRUs parameterize each hidden unit with $\lambda_k = r_k \exp(i\theta_k) = \exp(-\exp(\nu_k^{\log}) + i \exp(\theta_k^{\log}))$ and directly work with complex numbers. Consequently, the hidden layer cannot be directly used to predict real-values. It would be biased to take $\text{Re}(\bar{\mathbf{h}}_t)$ (see Appendix D.2), and instead an additional weight matrix $\bar{\mathbf{W}} \in \mathbb{C}^{n \times n}$ must be learned, to get $\text{Re}(\bar{\mathbf{W}} \bar{\mathbf{h}}_t)$. To understand why this works, assume that we took the original $\mathbf{h}_t$ from the dense NN, and handed it to an MLP. This would involve multiplying $\mathbf{W} \mathbf{h}_t$ for some $\mathbf{W}$. If we set $\bar{\mathbf{W}} = \mathbf{W} \mathbf{P}$, then $\bar{\mathbf{W}} \bar{\mathbf{h}}_t = \mathbf{W} \mathbf{P} \mathbf{P}^{-1} \mathbf{h}_t = \mathbf{W} \mathbf{h}_t$ and we did not introduce any bias. In fact, if $\bar{\mathbf{W}}$ is set this way, we do not need to take the real-valued part, because the output of $\bar{\mathbf{W}} \bar{\mathbf{h}}_t$ is real-valued. Of course, learning does not force this equivalence—in fact this parameterization is more flexible than the original—and so it is necessary to take the real-part.

RTUs avoid some of these complications by explicitly writing the recurrence and updates with real-valued hidden states. Implicitly, the relationship between the two real-valued hidden vectors forces

them to behave like complex numbers (as rotations), but all equations and learning stay in real-valued space. RTUs consequently avoid the need to post multiply by the matrix, removing a small number of learnable parameters, allowing the use of a nonlinear activation directly on the output, and allowing the hidden state to be immediately passed to a downstream MLP. We acknowledge that others may argue that working directly with complex numbers is preferable. The preference for real-valued hidden layers may simply be our own limitations, but we suspect much of the reinforcement learning community is similarly more comfortable to work in real-valued space. We found small choices in our implementation for LRU did not always behave as expected, partially due to how auto-differentiation is implemented in packages for complex numbers². In the end, our goal is to make these simple recurrent traces easy to use, and providing updates with real numbers may remove some barriers.

4 Online Prediction Learning

In this section, we explore different architectural variants of RTUs and LRUs in a online prediction task and then move on to study the tradeoffs between computational resources and performance when using RTUs with RTRL compared to GRUs and LRUs with T-BPTT.

4.1 Ablation Study on Architectural Choices for RTUs and LRUs

In this first experiment, we investigate the impact of several architectural choices on the performance of RTUs and LRUs varying where nonlinearity is applied. We use a simple multi-step prediction task called *Trace conditioning* [38] inspired by experiments in animal learning. The agent’s objective is to predict a signal—called the US—conditioned on an earlier signal—the CS. The prediction is formulated as a return, $G_t \doteq \sum_{k=0}^{\infty} \gamma^k \text{US}_{t+k+1}$, where the agent’s goal is to estimate the value function for this return. More details on this environment and experimental settings are in Appendix E. Figure 1 summarizes the results.

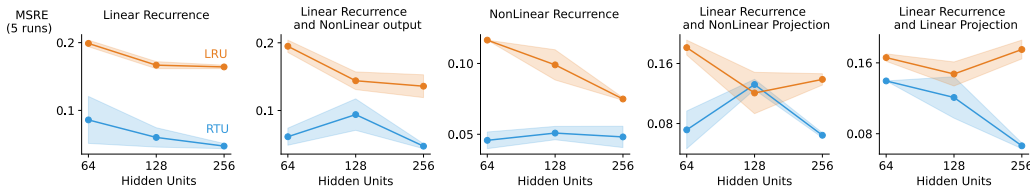


Figure 1: Ablation over different architectural choices for RTUs and LRUs. The RTU variants are blue, and the LRU variants are orange. In each subplot, we restrict both architectures in a particular way, reporting prediction error (MSRE) as a function of hidden state size. Across variations, RTUs are often better and, at worst, tie LRU. Here, both architectures were using RTRL.

4.2 Learning under resources constraints

In this section, we investigate the tradeoffs between computational resources and performance when using RTUs with RTRL compared to GRUs and LRUs with T-BPTT.

In the following experiments, all agents consist of a recurrent layer followed by a linear layer generating the prediction. We measure performance of the agents online by calculating the Mean Square Return Errors (MSRE), which is the mean squared error between the agent’s prediction at time t and G_t . In all the experiments, we used the Adam optimizer. We first ran each agent with different step sizes for 5 runs and 2 million steps. We then averaged the MSRE over the 2 million steps and selected each agent’s best step size value. Finally, we ran the agents with the best step size value for 10 runs, which we report here. We also report all agents’ step size sensitivity curves in Appendix E.3.

Learning under computational constraints: We first investigate: *how well do different agents exploit the available computational resources?* We specified a fixed computational budget of 15000

²Autodiff can give unexpected results when dealing with complex numbers. For example, see the discussion <https://github.com/google/jax/discussions/6817>.

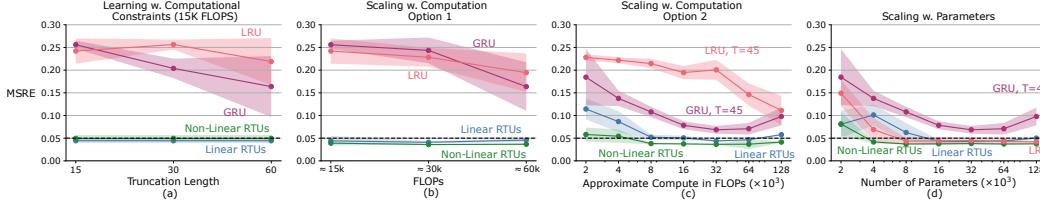


Figure 2: **Learning under resources constraints in Trace Conditioning.** Each of the four subplots shows how each algorithm’s performance varies as a function of resources. (a) LRU and GRU with T-TBTT is not competitive with RTUs even as T is increased while restricting the number of hidden units in LRU and GRU so that all algorithms use about the same computation per step. (b) If we allow GRU and LRU’s computation to increase (fixed network size) while increasing T , the performance gap remains. (c) Fixing T to a large value to solve the task, we can increase the number of parameters, holding the computation equal for all methods. (d) If we do not require compute to be equal across methods as we scale parameters, then the LRU can eventually match the error of RTU, but GRU cannot. The black dashed line represents the near perfect prediction performance.

FLOPs. Since RTUs are learning using RTRL and have a linear computational complexity, the computational budget only determines the number of hidden units in the architecture. For GRU and LRU, both the truncation length and the hidden dimension contribute to the budget. We tested several configurations of truncation lengths and parameters such that the overall computations fit the computational budget. Figure 2.a shows the results of this experiment. As we move along the horizontal axis, the number of parameters for GRU and LRU decreases as T increases to fit the computational constraints. However, the RTU agents do not depend on T , so their performance and computation is constant.

Scaling with computation: The computational complexity of T-BPTT depends on the truncation length and the number of parameters in the neural network. Thus, the agent can use the additional resources in two ways: (1) Increasing the truncation length, and (2) Increasing the number of parameters. On the other hand, RTUs use all the computations to have more parameters.

Now, we move to our second question: *how well do different methods scale with increasing the computational budget?* We answer this question in two stages: Firstly, we study T-BPTT with increasing T and a fixed number of parameters. For RTU, the computation increases by adding more parameters such that all corresponding points from GRU and RTU use the same amount of computation. Secondly, we fixed the truncation length for GRU to 45, which is more than the maximum distance between the CS and the US, and increased the computation by increasing the number of parameters for GRU. Again, for RTU, we increased the computation by increasing the number of parameters.

Figure of 2.b shows the first experiment’s results. While GRU’s performance improved as the truncation length increased, RTU outperformed GRU across all different computational budgets. Figure 2.c shows the results of the second experiment. The RTU agent’s performance consistently improves as we increase the computation available. However, the performance improvement for the GRU agent is inconsistent. The inconsistency of GRU performance could be associated with the trade-off between the truncation length and the number of parameters.

Scaling With Parameters: Finally, we study the performance of RTU and GRU when given the same number of parameters and allow the GRU agent to use more computation. We fixed the truncation length for GRU to 45 as before and used the same number of parameters for both agents. Figure 2.d shows the results of this experiment. For RTU, we see the same consistent performance improvement as we increase the number of parameters. For GRU, the performance improvement is also consistent, though it degrades slightly towards the end. The RTU agent outperforms the GRU agent even though the GRU uses more computation.

We provide additional experiments comparing RTUs to two other approaches that use RTRL: online LRUs and a real-valued diagonal RNN in Appendix E.1.

5 Real-Time Recurrent Policy Gradient

This section first highlights some differences in using *linear RTRL* methods, i.e., RTRL with linear complexity, in incremental and batch settings. We then investigate different ways of integrating linear

RTRL methods with policy gradient approaches, and we use PPO as a case study for this investigation. Finally, we compare the performance of RTRL methods with T-BPTT methods and other baselines.

5.1 Linear RTRL Methods in Incremental and Batch Settings

The benefits of linear RTRL methods over T-BPTT are more evident in the incremental rather than the batch setting. In the incremental learning setting, where the agent updates its parameters after each interaction step, linear RTRL methods have a constant computational complexity per update that depends only on the number of parameters. In contrast, T-BPTT methods have a complexity proportional to the truncation length T since T-BPTT methods require storing a sequence of past T activations to perform one gradient update. Figure 3 shows the time it takes to make one update with linear RTRL and T-BPTT given the same number of parameters. For T-BPTT, the time to make one update scales with the truncation length T , while for linear RTRL, it is constant.

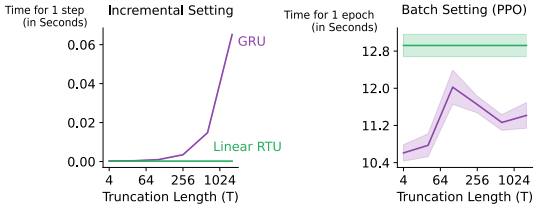


Figure 3: Contrasting runtime in incremental and batch settings. In the incremental setting, evaluated in the animal-learning prediction task, T-BPTT updates scale with truncation length, whereas linear RTRL is constant. With batch updates, evaluated in Ant-P with PPO, linear RTRL remains linear and T-BPTT is slightly more efficient.

The computational analysis for the batch setting is different than the incremental setting. In the batch setting, linear RTRL still have a constant cost per update and provide an untruncated yet stale gradient for all the samples. When using T-BPTT in the batch setting, there are two possibilities for the gradient updates. The first option, the typical strategy, is to divide the batch into non-overlapping sequences, each of length T , and perform T-BPTT on each sequence. In this case, the cost of one gradient update per sequence is a function of T , resulting in an effective update cost per sample independent of T . As a result, T-BPTT is computationally efficient in this case, albeit at the expense of a worse gradient estimate; in each sequence, only the last sample has a gradient estimate with T steps [24]. Figure 3 shows the time it takes to make one batch update with linear RTRL and T-BPTT given the same number of parameters. In this case, both methods use similar time per update. The second option is to divide the batch into overlapping sequences, where each gradient uses a sequence of T steps [24]. This approach increases the cost of updates per sample to be proportional to T , as in the incremental setting, with the benefit of better gradient estimates. However, all standard implementations of RL methods with T-BPTT use the computationally efficient option [37, 13, 23].

Integrating Linear RTRL Methods with PPO When performing batch updates, as with PPO, the RTRL gradients used to update the recurrent parameters will be stale, as they were calculated during the interaction with the environment w.r.t old policy and value parameters. One solution to mitigate the gradient staleness is to go through the whole trajectory after each epoch update and re-compute the gradient traces. However, this can be computationally expensive. In Appendix F, Algorithm 1, we provide the pseudocode for integrating RTRL methods with PPO with optional steps for re-running the network to update the RTRL gradient traces, the value targets, and the advantage estimates. We also performed an ablation study to investigate the effect of the gradient staleness in RTRL when combined with PPO, Appendix F. The results from the ablation study show that using a stale gradient results in better performance with RTUs and suggests that the staleness might help PPO maintain the trust region.

6 Experiments in Memory-Based Control

In this section, we evaluate the memory capabilities of RTUs when solving challenging RL control problems. We divide the problems in this section based on the source of partial observability. 1) Missing sensory data, where we mask out parts of the agent’s observation. The agent must accumulate and integrate the sensory observations over time to account for the missing information. 2) Remembering important cues, where the agent must remember an essential cue about the environment that happened many steps in advance.

Integrating Sensory Observations: We use the standard Mujoco POMDP benchmark widely used in prior work for evaluating memory-based RL agents [30, 11, 26, 31]. The benchmark consists of several challenging tasks where the agent controls a multi-joint dynamical body while only observing the joints’ positional (Mujoco-P) or velocity information (Mujoco-V). To increase experiment throughput, we use the Jax implementation of Mujoco from the Brax library [6] and implemented wrappers to mask either the velocity (Mujoco-P) or positional information (Mujoco-V).

We evaluated our Linear and Non-linear RTUs against GRU, LRU, and Online LRU. All agents use PPO [39] as the control algorithm, and the difference between the agents is the recurrent component. For all agents, we fixed the number of parameters for the recurrent part to be $\sim 24k$. We tuned the learning rate for all agents in all environments and selected the best learning rate for each agent per environment. We also included a GPT2-transformer baseline. We followed the implementation details in previous work [31], and used a GPT2 variant with 200k parameters. We provide the results for GPT2 in Appendix G.

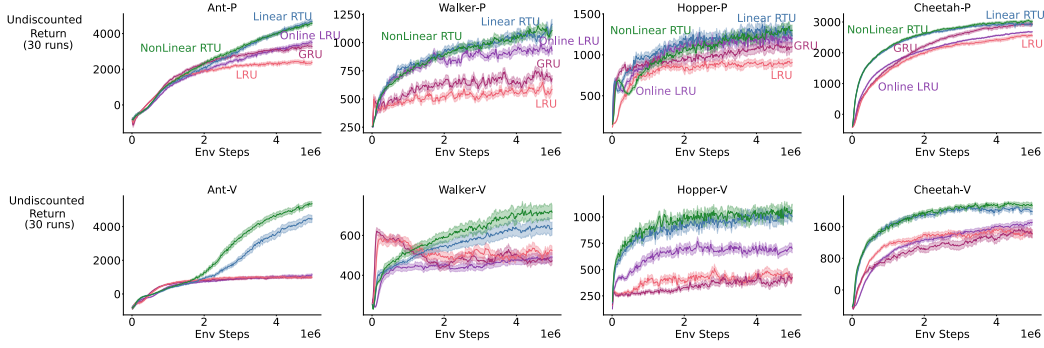


Figure 4: Learning curves on the Mujoco POMDP benchmark. Environments with -P mean that velocity components are occluded from the observations, while -V means that the positions and angles are occluded. All architectures have the same number of recurrent parameters (24k parameter). For each architecture, we show the performance of its best-tuned variant.

When given the same number of parameters, RTU agents outperform other baselines in all environments in Figure 4. Furthermore, we show in Appendix G that even when increasing the truncation length of both GRU and LRU agents to use significantly longer history, they outperform RTUs in only one task. Of particular note is again that RTUs outperform online LRUs, highlighting again that our simple modifications have a large impact on performance in this online RL setting.

Remembering Important Cues:

Next, we test the agents’ ability to remember essential environmental cues. We use several tasks from the POPGym benchmark [28] in addition to the Reacher POMDP task, a modified version of Mujoco Reacher where the agent observes the target position only at the beginning of the episode.

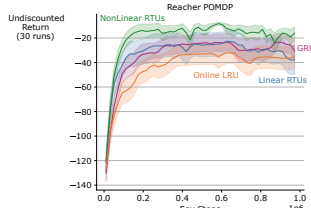


Figure 5: Reacher, 30 runs with standard errors.

In Reacher POMDP, GRU was able to achieve a similar performance to that of linear RTUs.

The POPGym tasks we consider along with the Reacher POMDP are all long-term memory tasks [31] as the agent must remember and carry the information for a long time.

Figure 5 summarizes the results for the reacher POMDP task and the POPGym results can be found in Figure 6. In both cases, we can see that RTUs outperform the other approaches. Non-linear RTUs achieve a better performance than linear RTUs in reacher POMDP, and both achieve a better performance in all tasks than online LRUs. In

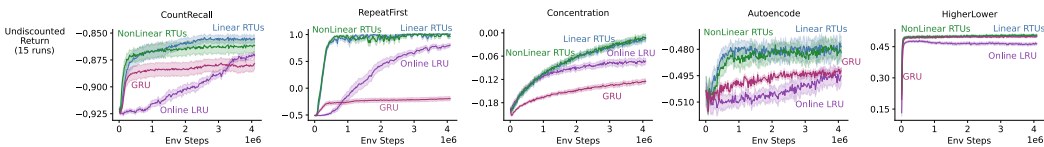


Figure 6: Results across several tasks from the POPGym benchmark.

7 Conclusion and Limitations

In this work, we investigated using complex-valued diagonal RNNs for online RL. We built on LRUs, to provide a small modification (RTUs) that we found performed significantly better in online RL across various partially observable prediction and control settings. We also found RTUs performed better than the more computationally intensive GRUs. Overall, RTUs are a promising, lightweight approach to online learning in partially observable RL environments.

A primary limitation of RTUs is the extension to multilayer recurrence. This limitation is inherent to all RTRL approaches; with multilayers, we need to save the gradient traces of the hidden state w.r.t the weights from all the preceding layers [16]. Previous work [16, 47] showed that using stop gradient operations between the layers and not tracing the time dependencies across layers is a viable choice. However, we need a more principled approach for tracing the gradient across layers.

One advantage of the linearity restriction in LRUs is that it allows the use of parallel scans for training [25]. However, recent works have shown the possibility of employing parallel scans to non-linear RNNs [7, 21]. A future direction is to investigate the use of parallel scans for training RTUs.

8 Acknowledgments

We would like to thank Nicolas Zucchet for advice about the online LRU implementation, and Subhojeet Pramanik for many discussions on transformers and RNNs. We would like to thank NSERC, CIFAR and Amii for research funding and the Digital Research Alliance of Canada for the computational resources.

References

- [1] Firas Al-Hafez, Guoping Zhao, Jan Peters, and Davide Tateo. Time-efficient reinforcement learning with stochastic stateful policies. In *The Twelfth International Conference on Learning Representations*.
- [2] Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of machine learning research*, 2018.
- [3] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [4] Tim Cooijmans and James Martens. On the variance of unbiased online recurrent optimization. *arXiv preprint arXiv:1902.02405*, 2019.
- [5] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In *International conference on machine learning*, 2018.
- [6] C. Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. Brax - a differentiable physics engine for large scale rigid body simulation, 2021.
- [7] Xavier Gonzalez, Andrew Warrington, Jimmy TH Smith, and Scott W Linderman. Towards scalable and stable parallelization of nonlinear rnns. *arXiv e-prints*, pages arXiv-2407, 2024.
- [8] Gori, Bengio, and De Mori. Bps: a learning algorithm for capturing the dynamic nature of speech. In *International Joint Conference on Neural Networks*, 1989.
- [9] Audrunas Gruslys, Will Dabney, Mohammad Gheshlaghi Azar, Bilal Piot, Marc Bellemare, and Remi Munos. The reactor: A fast and sample-efficient actor-critic agent for reinforcement learning. In *International Conference on Learning Representations*, 2018.
- [10] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [11] Dongqi Han, Kenji Doya, and Jun Tani. Variational recurrent models for solving partially observable control tasks. In *International Conference on Learning Representations*, 2019.

- [12] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. In *2015 aaai fall symposium series*, 2015.
- [13] Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and Jo  Go GM Ara    jo. Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 2022.
- [14] Kazuki Irie, Imanol Schlag, R  bert Csord  s, and J  rgen Schmidhuber. Going beyond linear transformers with recurrent fast weight programmers. *Advances in neural information processing systems*, 2021.
- [15] Kazuki Irie, Imanol Schlag, R  bert Csord  s, and J  rgen Schmidhuber. A modern self-referential weight matrix that learns to modify itself. In *International Conference on Machine Learning*, 2022.
- [16] Kazuki Irie, Anand Gopalakrishnan, and J  rgen Schmidhuber. Exploring the promise and limits of real-time recurrent learning, 2023.
- [17] Khurram Javed, Haseeb Shah, Rich Sutton, and Martha White. Online real-time recurrent learning using sparse connections and selective learning. *Journal of Machine Learning Research*, 2023.
- [18] Steven Kapturowski, Georg Ostrovski, Will Dabney, John Quan, and Remi Munos. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations*, 2019.
- [19] Shuai Li, Wanqing Li, Chris Cook, Ce Zhu, and Yanbo Gao. Independently Recurrent Neural Network (In-dRNN): Building A Longer and Deeper RNN. In *Conference on Computer Vision and Pattern Recognition*, 2018.
- [20] Xiujun Li, Lihong Li, Jianfeng Gao, Xiaodong He, Jianshu Chen, Li Deng, and Ji He. Recurrent reinforcement learning: a hybrid approach. *arXiv preprint arXiv:1509.03044*, 2015.
- [21] Yi Heng Lim, Qi Zhu, Joshua Selfridge, and Muhammad Firmansyah Kasim. Parallelizing non-linear sequential models over the sequence length. In *The Twelfth International Conference on Learning Representations*, 2023.
- [22] Chenhao Lu, Ruizhe Shi, Yuyao Liu, Kaizhe Hu, Simon Shaolei Du, and Huazhe Xu. Rethinking transformers in solving pomdps. In *Forty-first International Conference on Machine Learning*.
- [23] Chris Lu, Jakub Kuba, Alistair Letcher, Luke Metz, Christian Schroeder de Witt, and Jakob Foerster. Discovered policy optimisation. *Advances in Neural Information Processing Systems*, 2022.
- [24] Owen Marschall, Kyunghyun Cho, and Cristina Savin. A unified framework of online learning algorithms for training recurrent neural networks. *Journal of Machine Learning Research*, 2020.
- [25] Eric Martin and Chris Cundy. Parallelizing linear recurrent neural nets over sequence length. In *International Conference on Learning Representations*, 2018.
- [26] Lingheng Meng, Rob Gorbet, and Dana Kuli  . Memory-based deep reinforcement learning for pomdps. In *International Conference on Intelligent Robots and Systems (IROS)*, 2021.
- [27] Jacob Menick, Erich Elsen, Utku Evci, Simon Osindero, Karen Simonyan, and Alex Graves. Practical real time recurrent learning with a sparse approximation. In *International Conference on Learning Representations*, 2021.
- [28] Steven Morad, Ryan Kortvelesy, Matteo Bettini, Stephan Liwicki, and Amanda Prorok. POPGym: Benchmarking partially observable reinforcement learning. In *International Conference on Learning Representations*, 2023.
- [29] Michael Mozer. A focused backpropagation algorithm for temporal pattern recognition. *Complex Systems*, 1989.
- [30] Tianwei Ni, Benjamin Eysenbach, and Ruslan Salakhutdinov. Recurrent model-free rl can be a strong baseline for many pomdps. In *International Conference on Machine Learning*, 2022.
- [31] Tianwei Ni, Michel Ma, Benjamin Eysenbach, and Pierre-Luc Bacon. When do transformers shine in rl? decoupling memory from credit assignment. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [32] Yann Ollivier, Corentin Tallec, and Guillaume Charpiat. Training recurrent networks online without backtracking, 2015.

- [33] Antonio Orvieto, Samuel L Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences, 2023.
- [34] Emilio Parisotto, Francis Song, Jack Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant Jayakumar, Max Jaderberg, Raphael Lopez Kaufman, Aidan Clark, Seb Noury, et al. Stabilizing transformers for reinforcement learning. In *International conference on machine learning*, 2020.
- [35] P Ivan Pavlov. Conditioned reflexes: an investigation of the physiological activity of the cerebral cortex, 1927.
- [36] Subhojeet Pramanik, Esraa Elelimy, Marlos C Machado, and Adam White. Recurrent linear transformers. *arXiv preprint arXiv:2310.15719*, 2023.
- [37] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 2021.
- [38] Banafsheh Rafiee, Zaheer Abbas, Sina Ghiassian, Raksha Kumaraswamy, Richard S Sutton, Elliot A Ludvig, and Adam White. From eye-blinks to state construction: Diagnostic benchmarks for online representation learning. *Adaptive Behavior*, 2020.
- [39] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- [40] Sutton. Toward a new approach to model-based reinforcement learning, 2020.
- [41] Sutton. Markov and agent state, 2020.
- [42] Corentin Tallec and Yann Ollivier. Unbiased online recurrent optimization, 2017.
- [43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 2017.
- [44] Williams and Peng. An efficient gradient-based algorithm for on-line training of recurrent network trajectories. *Neural computation*, 1990.
- [45] Williams and Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1989.
- [46] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting?, 2022.
- [47] Nicolas Zucchet, Robert Meier, Simon Schug, Asier Mujika, and João Sacramento. Online learning of long range dependencies. In *Advances in Neural Information Processing Systems*, 2023.

A More Details on Representability with Complex-valued Diagonal Recurrence

This section explains why we need complex-valued diagonals to represent dense recurrent layers. We first show when it is equivalent to use complex-valued diagonal and a dense recurrent layer. We highlight that using a real-valued diagonal is like restricting the weights to be symmetric—because the (complex) diagonal corresponds to the eigenvalues of the weight matrix—which can severely limit representability. We provide a small experiment to show that complex eigenvalues naturally arise when training both a dense linear and nonlinear RNN, further motivating the utility of moving towards complex-valued diagonals.

A.1 Representability with Complex-valued Diagonals

Let us first consider when we can perfectly represent a dense, linear recurrent layer with a complex-valued diagonal recurrent layer. Assume we have the recurrence relationship, with learnable parameters $\mathbf{W}_h \in \mathbb{R}^{n \times n}$ and $\mathbf{W}_x \in \mathbb{R}^{n \times d}$,

$$\mathbf{h}_t \doteq f(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{u}(\mathbf{x}_t)) \quad (5)$$

where f is a potentially nonlinear function that inputs a vector and outputs the same-sized vector and $\mathbf{u}$ can be any transformation of the inputs $\mathbf{x}_t$ before they are inputted into the recurrent layer. The following equivalence result is straightforward but worthwhile formalizing.

Proposition A.1. Assume $f \circ \mathbf{P} = \mathbf{P} \circ f$ for any full rank, potentially complex-valued $\mathbf{P} \in \mathbb{C}^{n \times n}$ with unit-length column vectors. Then given any $\mathbf{W}_h$ and $\mathbf{W}_x$ for Equation (5), there is a corresponding complex-valued diagonal weight matrix $\mathbf{\Lambda} \in \mathbb{C}^{n \times n}$ and $\bar{\mathbf{W}}_x \in \mathbb{C}^{n \times d}$

$$\bar{\mathbf{h}}_t = f(\mathbf{\Lambda} \bar{\mathbf{h}}_{t-1} + \bar{\mathbf{W}}_x \mathbf{u}(\mathbf{x}_t)). \quad (6)$$

where $\bar{\mathbf{h}}_t \in \mathbb{C}^n$ is a linear transformation of $\mathbf{h}_t \in \mathbb{R}^n$.

Proof. We can rewrite the square matrix $\mathbf{W}_h$ using an eigenvalue decomposition $\mathbf{W}_h = \mathbf{P} \mathbf{\Lambda} \mathbf{P}^{-1}$, where $\mathbf{P}$ contains the n linearly independent eigenvectors and $\mathbf{\Lambda}$ is a diagonal matrix with the corresponding eigenvalues. Then, we can re-write (5) as:

$$\begin{aligned} \mathbf{h}_t &= f(\mathbf{P} \mathbf{\Lambda} \mathbf{P}^{-1} \mathbf{h}_{t-1} + \mathbf{P} \mathbf{P}^{-1} \mathbf{W}_x \mathbf{u}(\mathbf{x}_t)) \\ &= \mathbf{P} f(\mathbf{\Lambda} \mathbf{P}^{-1} \mathbf{h}_{t-1} + \mathbf{P}^{-1} \mathbf{W}_x \mathbf{u}(\mathbf{x}_t)) \\ \mathbf{P}^{-1} \mathbf{h}_t &= f(\mathbf{\Lambda} \mathbf{P}^{-1} \mathbf{h}_{t-1} + \mathbf{P}^{-1} \mathbf{W}_x \mathbf{u}(\mathbf{x}_t)) \end{aligned} \quad (7)$$

where $\mathbf{P}$ came outside of f under our assumption that it commutes with such matrices of eigenvectors. By defining $\bar{\mathbf{h}}_t \doteq \mathbf{P}^{-1} \mathbf{h}_t$ and $\bar{\mathbf{W}}_x \doteq \mathbf{P}^{-1} \mathbf{W}_x$, we get Eq. (6). $\square$

We can see $\bar{\mathbf{h}}_t$ and $\mathbf{h}_t$ are representationally equivalent: they are linearly weighted for downstream predictions, and so the linear transformation on $\bar{\mathbf{h}}_t$ can fold into this downstream linear weighting. But it is more computationally efficient to use $\bar{\mathbf{h}}_t$ with a diagonal weight matrix $\mathbf{\Lambda}$, meaning each hidden unit only has one recurrent relation instead of n .

Since we did not impose constraints on the matrix $\mathbf{W}_h$, other than being diagonalizable, the eigenvalues of $\mathbf{W}_h$ can be complex or real numbers. Previous diagonal RNNs such as eLSTM [16], Columnar networks [17], and IndRNN [19] use only real-valued diagonal matrices. Having only real-valued diagonals implicitly assumes that the matrix $\mathbf{W}_h$ is a symmetric matrix. [33] suggested using complex-valued diagonal matrices for better performance.

The above equivalence has only been used without any activation, namely in linear-recurrent units (LRUs) [33]. A natural question is if only the identity $f(\mathbf{x}) = \mathbf{x}$ (and linear functions) satisfy this property of commuting with eigenvector matrices. Intuitively, this seems like the only option, as imagining a nonlinearity that commutes is hard. Surprisingly, for the more restricted case of symmetric $\mathbf{W}_h$, we can show a slightly more general class of activations can be used, proving an if-and-only-if relationship (see Appendix A.2). However, even for this restricted setting, this generalized class is limited and such activations unlikely to be preferable to a linear recurrence. We see this as a negative result, that suggests this equivalence only holds for the linear setting.

A.2 More on the Equivalence of Non-Linear RTUs and Dense RNNs

As discussed in the main body, we likely only have an equivalence between using a full dense weight matrix and a complex-valued diagonal matrix for linear recurrent layers. However, we can obtain a slightly more general equivalence in the restricted setting where the weight matrix for the recurrence is symmetric. This restricted setting is not of general interest, but we include the result here because it could be of interest to a few.

For the case where we have a symmetric weight matrix, we need the activation to commute with orthonormal matrices. Consider again the form

$$\mathbf{h}_t \doteq f(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t) \quad (5)$$

where f is a potentially nonlinear function that inputs a vector and outputs the same-sized vector. To obtain the equivalence, assume that for any orthonormal matrix $\mathbf{A}$, $f \circ \mathbf{A} = \mathbf{A} \circ f$. We can rewrite the square and symmetric matrix $\mathbf{W}_h$ using an eigenvalue decomposition $\mathbf{W}_h = \mathbf{A} \mathbf{\Lambda} \mathbf{A}^\top$, where $\mathbf{A}$ contains the n linearly independent eigenvectors and is an orthonormal matrix and $\mathbf{\Lambda}$ is a diagonal matrix with the corresponding eigenvalues. Then, we can re-write (5) as:

$$\begin{aligned} \mathbf{h}_t &= f(\mathbf{A} \mathbf{\Lambda} \mathbf{A}^\top \mathbf{h}_{t-1} + \mathbf{A} \mathbf{A}^\top \mathbf{W}_x \mathbf{x}_t) \\ &= \mathbf{A} f(\mathbf{\Lambda} \mathbf{A}^\top \mathbf{h}_{t-1} + \mathbf{A}^\top \mathbf{W}_x \mathbf{x}_t) \\ \mathbf{A}^\top \mathbf{h}_t &= \mathbf{\Lambda} \mathbf{A}^\top \mathbf{h}_{t-1} + \mathbf{A}^\top \mathbf{W}_x \mathbf{x}_t \end{aligned} \quad (8)$$

where $\mathbf{A}$ came outside of f because commutes with orthonormal matrices. By defining $\bar{\mathbf{h}}_t \doteq \mathbf{A}^\top \mathbf{h}_t$ and $\bar{\mathbf{W}}_x \doteq \mathbf{A}^\top \mathbf{W}_x$, we get:

$$\bar{\mathbf{h}}_t = f(\Lambda \bar{\mathbf{h}}_{t-1} + \bar{\mathbf{W}}_x \mathbf{x}_t). \quad (6)$$

Each hidden unit now has one recurrent relation instead of n , because our weight matrix Λ is diagonal.

A natural question is if only the identity $f(\mathbf{x}) = \mathbf{x}$ —namely linear recurrence—satisfies this property of commuting with orthonormal matrices. We show below that it holds for a slightly more general class of recurrent layers, proving an if-and-only-if relationship. We see the below result as a negative result, highlighting that this equivalence largely only holds for the linear setting and does not generalize to other activations of interest. It provides even further evidence that likely the only setting of interest for the general non-symmetric case is also with a linear recurrence.

Nonetheless, let us obtain the if-and-only-if for completeness. A simple extension that continues to satisfies this property is $f(\mathbf{x}) = \mathbf{x}c(\|\mathbf{x}\|_2)$ for any $c : \mathbb{R} \rightarrow \mathbb{R}$. We can see that for any orthonormal $\mathbf{A}$, we have

$$f(\mathbf{Ax}) = \mathbf{Ax}c(\|\mathbf{Ax}\|_2) = \mathbf{Ax}c(\|\mathbf{x}\|_2) = \mathbf{A}f(\mathbf{x}).$$

This means that we can have activations that rescale the input $\mathbf{x}$ depending on the norm of that input.

More generally, the activation can involve matrices and rotations. We can define $\boldsymbol{\theta}(\mathbf{x}) = [\mathbf{x} \mathbf{U}(\mathbf{x})]^\top$ where $\mathbf{U}(\mathbf{x}) \in \mathbb{R}^{n \times n-1}$ is a matrix where the columns are orthogonal vectors to each other and to $\mathbf{x}$. Then for any vector-valued $g : \mathbb{R} \rightarrow \mathbb{R}^n$, we have that $f(\mathbf{x}) = \boldsymbol{\theta}(\mathbf{x})^\top g(\|\mathbf{x}\|_2)$ also satisfies this property:

$$\begin{aligned} f(\mathbf{Ax}) &= \boldsymbol{\theta}(\mathbf{Ax})^\top g(\|\mathbf{Ax}\|_2) = [\mathbf{Ax}; \mathbf{U}(\mathbf{Ax})]^\top g(\|\mathbf{x}\|_2) \\ &= \mathbf{A}[\mathbf{x}; \mathbf{U}(\mathbf{x})]^\top g(\|\mathbf{x}\|_2) = \mathbf{A}f(\mathbf{x}). \end{aligned}$$

The last line follows because $\mathbf{AU}(\mathbf{x}) = \mathbf{U}(\mathbf{Ax})$. Namely, for any orthogonal vector $\mathbf{u}$ with $\mathbf{u}^\top \mathbf{x} = 0$, we have that $\tilde{\mathbf{u}} \doteq \mathbf{Au}$ satisfies $\tilde{\mathbf{u}}^\top \mathbf{Ax} = \mathbf{u}^\top \mathbf{A}^\top \mathbf{Ax} = \mathbf{u}^\top \mathbf{x} = 0$ because $\mathbf{A}^\top \mathbf{A} = \mathbf{I}$.

Now we show this formally. Denote $O^n \subset \mathbb{R}^{n \times n} : \mathbf{A} \in O^n \iff \mathbf{A}^\top \mathbf{A} = \mathbf{AA}^\top = \mathbf{I}$. Denote

$$e_i = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \text{ where the } i^{th} \text{ element is 1.}$$

Definition A.2. $\boldsymbol{\theta}(\mathbf{x})$ is a matrix such that:

$$\begin{aligned} \boldsymbol{\theta}(\mathbf{x}) &\in O^n \\ \boldsymbol{\theta}(\mathbf{x})\mathbf{x} &= \|\mathbf{x}\|_2^2 e_1 \end{aligned} \quad (9)$$

Lemma A.3. $\boldsymbol{\theta}(\mathbf{x})$ exists and $\forall \mathbf{A} \in O^n, \boldsymbol{\theta}(\mathbf{Ax}) = \boldsymbol{\theta}(\mathbf{x})\mathbf{A}^\top$

Proof. Let $\boldsymbol{\theta}(\mathbf{x})$ be $\in O^n$. Then, by definition of O^n , $\boldsymbol{\theta}(\mathbf{x})^\top \boldsymbol{\theta}(\mathbf{x}) = \boldsymbol{\theta}(\mathbf{x})\boldsymbol{\theta}(\mathbf{x})^\top = \mathbf{I}$. Let $\boldsymbol{\theta}(\mathbf{x})[1] = \mathbf{x}$, and $\boldsymbol{\theta}(\mathbf{x})[2:]$ be any orthogonal vectors, also orthogonal to $\mathbf{x}$. Where $\boldsymbol{\theta}(\mathbf{x})[i]$ be the i^{th} row of $\boldsymbol{\theta}(\mathbf{x})$. Then, for $\mathbf{x} \in \mathbb{R}^n$, $\boldsymbol{\theta}(\mathbf{x})\mathbf{x} = \|\mathbf{x}\|_2^2 e_1$.

$$\begin{aligned} \boldsymbol{\theta}(\mathbf{Ax})\mathbf{Ax} &= \|\mathbf{Ax}\|_2^2 e_1 = \|\mathbf{x}\|_2^2 e_1 = \boldsymbol{\theta}(\mathbf{x})\mathbf{x} \\ \boldsymbol{\theta}(\mathbf{Ax})^\top \boldsymbol{\theta}(\mathbf{Ax})\mathbf{Ax} &= \boldsymbol{\theta}(\mathbf{Ax})^\top \boldsymbol{\theta}(\mathbf{x})\mathbf{x} \\ \mathbf{Ax} &= \boldsymbol{\theta}(\mathbf{Ax})^\top \boldsymbol{\theta}(\mathbf{x})\mathbf{x} \\ \mathbf{A}^\top \mathbf{Ax} &= \mathbf{A}^\top \boldsymbol{\theta}(\mathbf{Ax})^\top \boldsymbol{\theta}(\mathbf{x})\mathbf{x} \\ \mathbf{x} &= \mathbf{A}^\top \boldsymbol{\theta}(\mathbf{Ax})^\top \boldsymbol{\theta}(\mathbf{x})\mathbf{x} \\ \mathbf{A}^\top \boldsymbol{\theta}(\mathbf{Ax})^\top \boldsymbol{\theta}(\mathbf{x}) &= \mathbf{I} \\ \boldsymbol{\theta}(\mathbf{Ax})^\top &= \boldsymbol{\theta}(\mathbf{x})\mathbf{A}^\top \end{aligned} \quad (10)$$

□

Theorem A.4. Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ and $A \in O^n$. Then, $f \circ A = A \circ f \iff \exists g : \mathbb{R} \rightarrow \mathbb{R}^n, f(x) = \theta(x)^T g(\|x\|_2)$

Proof. Define $g(\alpha) = f(\alpha^2 e_1)$.

$$\begin{aligned} \mathbf{f}(\mathbf{x}) &= \boldsymbol{\theta}(\mathbf{x})^T \boldsymbol{\theta}(\mathbf{x}) \mathbf{f}(\mathbf{x}) \\ &= \boldsymbol{\theta}(\mathbf{x})^T \mathbf{f}(\boldsymbol{\theta}(\mathbf{x}) \mathbf{x}) \\ &= \boldsymbol{\theta}(\mathbf{x})^T \mathbf{f}(\|\mathbf{x}\|_2^2 \mathbf{e}_1) \\ &= \boldsymbol{\theta}(\mathbf{x})^T \mathbf{g}(\|\mathbf{x}\|_2) \end{aligned} \tag{11}$$

$$\begin{aligned} (\mathbf{f} \circ \mathbf{A})(\mathbf{x}) &= \mathbf{f}(\mathbf{A}\mathbf{x}) \\ &= \boldsymbol{\theta}(\mathbf{A}\mathbf{x})^T \mathbf{g}(\|\mathbf{A}\mathbf{x}\|_2) \\ &= \boldsymbol{\theta}(\mathbf{A}\mathbf{x})^T \mathbf{g}(\|\mathbf{x}\|_2) \\ &= \mathbf{A}\boldsymbol{\theta}(\mathbf{x})^T \mathbf{g}(\|\mathbf{x}\|_2) \\ &= (\mathbf{A} \circ \mathbf{f})(\mathbf{x}) \end{aligned} \tag{12}$$

□

A.3 Complex Eigenvalues in Vanilla RNNs

We empirically investigate whether complex eigenvalues appear when training dense RNNs in a simple task. The goal is to show that $\mathbf{W}_h$ is not a symmetric matrix, even in the simplest tasks, so having only real-valued diagonals is too restrictive. We used the Three State POMDP[41] for this experiment.

The Three State MDP is depicted in Figure 7. In this task, the agent needs to remember one cue from the previous time-step ago, to make a prediction about the next time-step. The MDP has three states, s_1 , s_2 , and s_3 , and no actions. If the agent is in either s_1 or s_2 , it transitions to any of the three states with equal probability. However, if the agent is in s_3 , it transitions to the state preceded by s_3 . A sequence of observations would look like 1, 3, 1, 2, 2, 3, 2, $\dots$, and we ask the agent to predict the next observation.

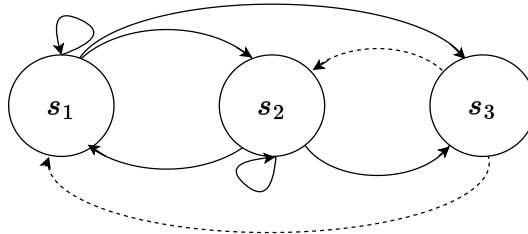


Figure 7: Illustration of the Three State MDP. We used dashed lines for the transitions starting in s_3 to make them more visible.

We trained a vanilla RNN with 3 hidden states with T-BPTT with truncation length 2, which is a sufficient history length in this problem to predict the next observation. Since we have 3 hidden states, the matrix $\mathbf{W}_h$ is $\in \mathbb{R}^{3 \times 3}$ and could have at most 2 complex eigenvalues.

We measured the performance in terms of the percentage of correct predictions made in S_3 . We recorded the number of complex eigenvalues of $\mathbf{W}_h$ after each parameter update, shown in Figure 8. This agent reaches 100% accuracy in this problem relatively quickly. We can also see that the agent oscillates between having two complex eigenvalues and zero eigenvalues. The average number of complex eigenvalues across 30 run is above 1.5, which means that on more than $\frac{3}{4}$ of the steps, the RNN has two complex eigenvalues. The primary point is that we see complex eigenvalues appear frequently.

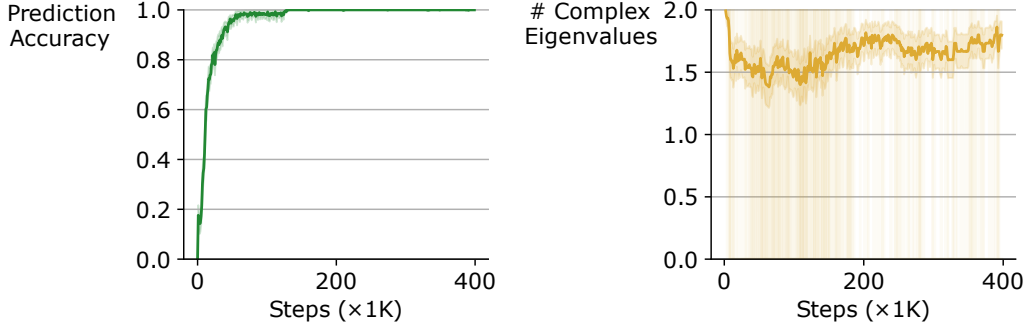


Figure 8: **Left:** The percentage of correct predictions when training an RNN in the Three State MDP. **Right:** Number of complex eigenvalues when training an RNN in the Three States MDP. The solid line is the mean over 30 runs, the shaded region area is the standard error, and the lines are individual runs.

B Background on BackPropagation Through Time and Real-Time Recurrent Learning

This section provides a brief background on BackPropagation Through Time (BPTT) and Real-Time Recurrent Learning (RTRL) algorithms.

B.1 BackPropagation Through Time

BPTT calculates the gradient, $\nabla_{\psi} \mathcal{L}$, by unfolding the recurrent dynamics through time and incorporating the impact of the parameters on the loss from all observed time steps. Formally, we can write $\nabla_{\psi} \mathcal{L}$ as:

$$\nabla_{\psi} \mathcal{L} = \frac{1}{t} \sum_{i=0}^{t-1} \nabla_{\psi} \mathcal{L}_i. \quad (13)$$

Applying the chain rule, we re-write Eq.13 as:

$$\begin{aligned} \nabla_{\psi} \mathcal{L} &= \frac{1}{t} \sum_{i=0}^{t-1} \nabla_{\psi} \mathcal{L}_i \\ &= \frac{1}{t} \sum_{i=0}^{t-1} \frac{\partial \mathcal{L}_i}{\partial \mathbf{h}_i} \frac{\partial \mathbf{h}_i}{\partial \psi}. \end{aligned} \quad (14)$$

When calculating $\frac{\partial \mathbf{h}_i}{\partial \psi}$, we need to consider the effect of ψ from all the time steps. To illustrate this effect, consider unrolling the last 2 steps of the RNN dynamics:

$$\begin{aligned} \mathbf{h}_t &= \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \psi) \\ \text{Re-write } \mathbf{h}_{t-1} &\text{ as } \mathbf{f}(\mathbf{h}_{t-2}, \mathbf{x}_{t-1}, \psi) \\ &= \mathbf{f}(\mathbf{f}(\mathbf{h}_{t-2}, \mathbf{x}_{t-1}, \psi), \mathbf{x}_t, \psi) \\ \text{Re-write } \mathbf{h}_{t-2} &\text{ as } \mathbf{f}(\mathbf{h}_{t-3}, \mathbf{x}_{t-2}, \psi) \\ &= \mathbf{f}(\mathbf{f}(\mathbf{f}(\mathbf{h}_{t-3}, \mathbf{x}_{t-2}, \psi), \mathbf{x}_{t-1}, \psi), \mathbf{x}_t, \psi). \end{aligned} \quad (15)$$

Equation 15 shows that the network parameters ψ affect the construction of the recurrent state $\mathbf{h}_t$ through two pathways: a direct pathway, i.e., using ψ to evaluate $\mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \psi)$, and an implicit pathway, i.e., ψ affected constructing all previous recurrent states, $\mathbf{h}_{t-1}, \dots, \mathbf{h}_1$, and all those recurrent states affected $\mathbf{h}_t$ construction. Thus, to calculate $\frac{\partial \mathbf{h}_t}{\partial \psi}$, we need to consider those two pathways:

$$\frac{\partial \mathbf{h}_t}{\partial \psi} = \frac{\partial \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \psi)}{\partial \psi} + \frac{\partial \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \psi)}{\partial \mathbf{h}_{t-1}} \frac{\partial \mathbf{h}_{t-1}}{\partial \psi}. \quad (16)$$

Once again, we need to consider the two pathways when evaluating $\frac{\partial \mathbf{h}_{t-1}}{\partial \boldsymbol{\psi}}$ in 16. For simplicity, let $\mathbf{J}_t \doteq \frac{\partial \mathbf{h}_t}{\partial \boldsymbol{\psi}}$, $\mathbf{B}_t = \frac{\partial \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \boldsymbol{\psi})}{\partial \boldsymbol{\psi}}$, $\mathbf{C}_t = \frac{\partial \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \boldsymbol{\psi})}{\partial \mathbf{h}_{t-1}}$, and re-write 16:

$$\begin{aligned}
\mathbf{J}_t &= \mathbf{B}_t + \mathbf{C}_t \mathbf{J}_{t-1} \\
&= \mathbf{B}_t + \mathbf{C}_t (\mathbf{B}_{t-1} + \mathbf{C}_{t-1} \mathbf{J}_{t-2}) \quad \text{Unrolling } \mathbf{J}_{t-1} \\
&= \mathbf{B}_t + \mathbf{C}_t \mathbf{B}_{t-1} + \mathbf{C}_t \mathbf{C}_{t-1} \mathbf{J}_{t-2} \\
&= \mathbf{B}_t + \mathbf{C}_t \mathbf{B}_{t-1} + \mathbf{C}_t \mathbf{C}_{t-1} (\mathbf{B}_{t-2} + \mathbf{C}_{t-2} \mathbf{J}_{t-3}) \quad \text{Unrolling } \mathbf{J}_{t-2} \\
&= \mathbf{B}_t + \mathbf{C}_t \mathbf{B}_{t-1} + \mathbf{C}_t \mathbf{C}_{t-1} \mathbf{B}_{t-2} + \mathbf{C}_t \mathbf{C}_{t-1} \mathbf{C}_{t-2} \mathbf{J}_{t-3} \\
&= \mathbf{B}_t + \mathbf{C}_t \mathbf{B}_{t-1} + \mathbf{C}_t \mathbf{C}_{t-1} \mathbf{B}_{t-2} + \dots + \mathbf{C}_t \mathbf{C}_{t-1} \mathbf{C}_{t-2} \dots \mathbf{C}_2 \mathbf{B}_1 + \mathbf{C}_t \mathbf{C}_{t-1} \mathbf{C}_{t-2} \dots \mathbf{C}_1 \mathbf{J}_0 \quad \text{Keep unrolling} \\
&= \sum_{k=1}^t \left(\prod_{i=k+1}^t \mathbf{C}_i \right) \mathbf{B}_k + \left(\prod_{i=1}^t \mathbf{C}_i \right) \mathbf{J}_0.
\end{aligned} \tag{17}$$

Writing $\frac{\partial \mathbf{h}_t}{\partial \boldsymbol{\psi}}$ using the results from 17:

$$\begin{aligned}
\frac{\partial \mathbf{h}_t}{\partial \boldsymbol{\psi}} &= \frac{\partial \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \boldsymbol{\psi})}{\partial \boldsymbol{\psi}} + \frac{\partial \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \boldsymbol{\psi})}{\partial \mathbf{h}_{t-1}} \frac{\partial \mathbf{h}_{t-1}}{\partial \boldsymbol{\psi}} \\
&= \sum_{k=1}^t \left(\prod_{i=k+1}^t \frac{\partial \mathbf{f}(\mathbf{h}_{i-1}, \mathbf{x}_i, \boldsymbol{\psi})}{\partial \mathbf{h}_{i-1}} \right) \frac{\partial \mathbf{f}(\mathbf{h}_{k-1}, \mathbf{x}_k, \boldsymbol{\psi})}{\partial \boldsymbol{\psi}} + \left(\prod_{i=1}^t \frac{\partial \mathbf{f}(\mathbf{h}_{i-1}, \mathbf{x}_i, \boldsymbol{\psi})}{\partial \mathbf{h}_{i-1}} \right) \frac{\partial \mathbf{h}_0}{\partial \boldsymbol{\psi}}.
\end{aligned} \tag{18}$$

According to Eq. 18, the agent needs to store all the previous inputs to calculate $\frac{\partial \mathbf{h}_t}{\partial \boldsymbol{\psi}}$ which is impractical; the computation and memory complexity will be increasing with t .

B.1.1 Truncated-BackPropagation Through Time

Williams and Peng (1990) introduced Truncated-BackPropagation Through Time (T-BPTT) which solves the issue of increasing memory and computational complexities of BPTT. In T-BPTT, we specify a truncation length T , which controls the number of steps considered when calculating the gradient in 18. We now write the truncated version of 17 which takes into consideration the gradient from the last T steps only:

$$\mathbf{J}_t = \sum_{k=t-T}^t \left(\prod_{i=k+1}^t \mathbf{C}_i \right) \mathbf{B}_k \tag{19}$$

Using results from 19, we then write the approximated gradient of the loss w.r.t the learnable parameters:

$$\begin{aligned}
\nabla_{\boldsymbol{\psi}} \mathcal{L} &= \sum_{j=t-T}^t \frac{\partial \mathcal{L}_j}{\partial \mathbf{h}_j} \frac{\partial \mathbf{h}_j}{\partial \boldsymbol{\psi}} \\
&= \sum_{j=t-T}^t \frac{\partial \mathcal{L}_j}{\partial \mathbf{h}_j} \sum_{k=j-T}^j \left(\prod_{i=k+1}^j \frac{\partial \mathbf{f}(\mathbf{h}_{i-1}, \mathbf{x}_i, \boldsymbol{\psi})}{\partial \mathbf{h}_{i-1}} \right) \frac{\partial \mathbf{f}(\mathbf{h}_{k-1}, \mathbf{x}_k, \boldsymbol{\psi})}{\partial \boldsymbol{\psi}} + \left(\prod_{i=1}^j \frac{\partial \mathbf{f}(\mathbf{h}_{i-1}, \mathbf{x}_i, \boldsymbol{\psi})}{\partial \mathbf{h}_{i-1}} \right) \frac{\partial \mathbf{h}_0}{\partial \boldsymbol{\psi}}
\end{aligned} \tag{20}$$

B.2 Real-Time Recurrent Learning

Williams and Zipser (1989) introduced the Real-time Recurrent Learning algorithm (RTRL) as a learning algorithm for continual recurrent learning. RTRL employs the recurrent formulation of the gradient in 16; instead of unrolling $\frac{\partial \mathbf{h}_{t-1}}{\partial \boldsymbol{\psi}}$ further back in time, RTRL saves its calculated value from the previous time step and use it later when needed. It is worth emphasizing that after the agent updates its parameters, the gradient information saved from previous time steps would be stale, i.e., calculated w.r.t old parameters, however, under the assumption of small learning rates, RTRL is

known to converge. The gradient formulation of RTRL can be written as:

$$\begin{aligned}
\nabla_{\psi} \mathcal{L} &= \sum_{i=0}^t \frac{\partial \mathcal{L}_i}{\partial \mathbf{h}_i} \frac{\partial \mathbf{h}_i}{\partial \psi} \\
&= \sum_{i=0}^t \frac{\partial \mathcal{L}_i}{\partial \mathbf{h}_i} \left(\frac{\partial \mathbf{f}(\mathbf{h}_{i-1}, \mathbf{x}_i, \psi)}{\partial \psi} + \frac{\partial \mathbf{f}(\mathbf{h}_{i-1}, \mathbf{x}_i, \psi)}{\partial \mathbf{h}_{i-1}} \frac{\partial \mathbf{h}_{i-1}}{\partial \psi} \right)
\end{aligned} \tag{21}$$

C Recurrent Trace Units

This appendix details the parametrization used for RTUs, the derivation of the RTRL update rules, and the extension of RTUs to multi-layers.

C.1 Empirical Analysis for different r and θ parameterizations:

Since r represents the magnitude of a complex number, then $r \in \mathbb{R}^+$. Additionally, it is preferred to have $r \in (0, 1]$ to avoid vanishing/exploding gradients. Let w_r be a learnable parameter which could directly represent r or represent a function of r , we can enforce the constraints on r in several ways:

1. Direct learning: Learn r directly, and clip it after each parameter's update to be in $(0, 1]$.
2. Enforcing $r \in \mathbb{R}^+$: Learn ν such that $r \doteq \exp(-\nu)$. This parameterization enforces r to be positive. However, additional clipping is needed to enforce $r \in (0, 1]$.
3. Enforcing stability on r : We can enforce r to be $\in (0, 1]$ by using a positive non-linear function. For example, learn $\nu^{\log}$ such that $r \doteq \exp(-\exp(\nu^{\log}))$, this parameterization ensures that $r \in (0, 1]$ and is suggested by Orvieto et al. [33]. Another example is learning $r = \sigma(\nu)$, which also ensures $r \in (0, 1]$.

Finally, we can also enforce stability on θ by learning $\theta^{\log}$ such that $\theta \doteq \exp(\theta^{\log})$ to ensure that θ is always positive.

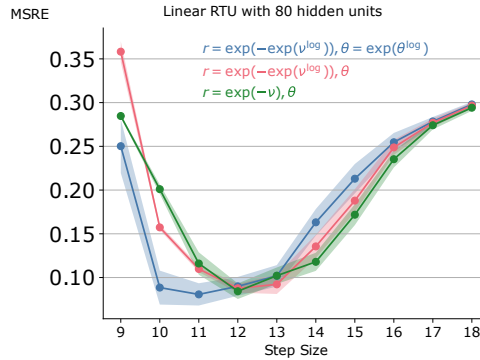


Figure 9: Learning rate sensitivity for different parameterizations of r and θ for RTUs with 80 hidden units.

We empirically compare the different parameterizations of r and θ . In our experiments, learning r directly resulted in unstable training where the MSRE diverges. We plot the learning rate sensitivity for the other parameterization for two different sizes of RTUs in Figures 9 and 10. While all the parametrizations produce similar performance, we notice that learning $\theta^{\log}$ and $\nu^{\log}$ has better learning rate sensitivity.

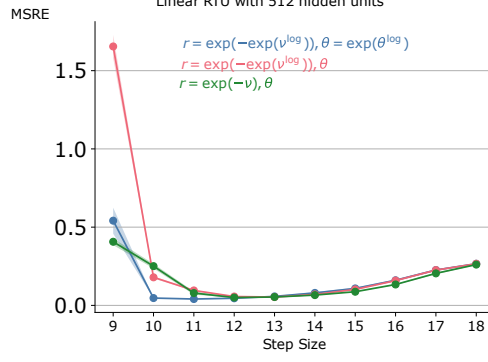


Figure 10: Learning rate sensitivity for different parameterizations of r and θ for RTUs with 512 hidden units.

C.2 Real-Time Recurrent Learning for a Single Layer Linear RTUs

We now outline the details for the RTRL update rules for RTUs. The set of learnable parameters for RTUs is $\psi = \{\nu^{\log}, \theta^{\log}, \mathbf{W}_x^{c1}, \mathbf{W}_x^{c2}\}$. At each time step t , the learner receives a loss $\mathcal{L}_t(\hat{y}_t, y_t; \psi)$ where y_t is the network output at time t , then the gradient of the loss w.r.t the parameters is:

$$\frac{\partial \mathcal{L}_t}{\partial \psi} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_t^{c1}} \frac{\partial \mathbf{h}_t^{c1}}{\partial \psi} + \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_t^{c2}} \frac{\partial \mathbf{h}_t^{c2}}{\partial \psi}, \quad (22)$$

where $\frac{\partial \mathbf{h}_t^{c1}}{\partial \psi} = \left\{ \frac{\partial \mathbf{h}_t^{c1}}{\partial \nu^{\log}}, \frac{\partial \mathbf{h}_t^{c1}}{\partial \theta^{\log}}, \frac{\partial \mathbf{h}_t^{c1}}{\partial \mathbf{W}_x^{c1}}, \frac{\partial \mathbf{h}_t^{c1}}{\partial \mathbf{W}_x^{c2}} \right\}$ and $\frac{\partial \mathbf{h}_t^{c2}}{\partial \psi} = \left\{ \frac{\partial \mathbf{h}_t^{c2}}{\partial \nu^{\log}}, \frac{\partial \mathbf{h}_t^{c2}}{\partial \theta^{\log}}, \frac{\partial \mathbf{h}_t^{c2}}{\partial \mathbf{W}_x^{c2}}, \frac{\partial \mathbf{h}_t^{c2}}{\partial \mathbf{W}_x^{c1}} \right\}$.

We can derive the following gradients:

$$\begin{aligned} \frac{\partial \mathbf{h}_t^{c1}}{\partial \nu^{\log}} &= \frac{\partial \mathbf{g}(\nu, \theta)}{\partial \nu^{\log}} \odot \mathbf{h}_{t-1}^{c1} + \mathbf{g}(\nu, \theta) \frac{\partial \mathbf{h}_{t-1}^{c1}}{\partial \nu^{\log}} - \frac{\partial \phi(\nu, \theta)}{\partial \nu^{\log}} \odot \mathbf{h}_{t-1}^{c2} - \phi(\nu, \theta) \frac{\partial \mathbf{h}_{t-1}^{c2}}{\partial \nu^{\log}} + \frac{\partial \gamma}{\partial \nu^{\log}} \odot \mathbf{W}_x^{c1} \mathbf{x}_t \\ \frac{\partial \mathbf{h}_t^{c2}}{\partial \nu^{\log}} &= \frac{\partial \mathbf{g}(\nu, \theta)}{\partial \nu^{\log}} \odot \mathbf{h}_{t-1}^{c2} + \mathbf{g}(\nu, \theta) \frac{\partial \mathbf{h}_{t-1}^{c2}}{\partial \nu^{\log}} + \frac{\partial \phi(\nu, \theta)}{\partial \nu^{\log}} \odot \mathbf{h}_{t-1}^{c1} + \phi(\nu, \theta) \frac{\partial \mathbf{h}_{t-1}^{c1}}{\partial \nu^{\log}} + \frac{\partial \gamma}{\partial \nu^{\log}} \odot \mathbf{W}_x^{c2} \mathbf{x}_t \end{aligned} \quad (23)$$

$$\begin{aligned} \frac{\partial \mathbf{h}_t^{c1}}{\partial \theta^{\log}} &= \frac{\partial \mathbf{g}(\nu, \theta)}{\partial \theta^{\log}} \odot \mathbf{h}_{t-1}^{c1} + \mathbf{g}(\nu, \theta) \frac{\partial \mathbf{h}_{t-1}^{c1}}{\partial \theta^{\log}} - \frac{\partial \phi(\nu, \theta)}{\partial \theta^{\log}} \odot \mathbf{h}_{t-1}^{c2} - \phi(\nu, \theta) \frac{\partial \mathbf{h}_{t-1}^{c2}}{\partial \theta^{\log}} \\ \frac{\partial \mathbf{h}_t^{c2}}{\partial \theta^{\log}} &= \frac{\partial \mathbf{g}(\nu, \theta)}{\partial \theta^{\log}} \odot \mathbf{h}_{t-1}^{c2} + \mathbf{g}(\nu, \theta) \frac{\partial \mathbf{h}_{t-1}^{c2}}{\partial \theta^{\log}} + \frac{\partial \phi(\nu, \theta)}{\partial \theta^{\log}} \odot \mathbf{h}_{t-1}^{c1} + \phi(\nu, \theta) \frac{\partial \mathbf{h}_{t-1}^{c1}}{\partial \theta^{\log}} \end{aligned} \quad (24)$$

where

$$\begin{aligned} \frac{\partial \mathbf{g}(\nu, \theta)}{\partial \nu^{\log}} &= -\mathbf{g}(\nu, \theta) \exp(\nu^{\log}) \\ \frac{\partial \mathbf{g}(\nu, \theta)}{\partial \theta^{\log}} &= -\phi(\nu, \theta) \exp(\theta^{\log}) \\ \frac{\partial \phi(\nu, \theta)}{\partial \nu^{\log}} &= -\phi(\nu, \theta) \exp(\nu^{\log}) \\ \frac{\partial \phi(\nu, \theta)}{\partial \theta^{\log}} &= \mathbf{g}(\nu, \theta) \exp(\theta^{\log}) \end{aligned} \quad (25)$$

To efficiently compute the gradient w.r.t $\mathbf{W}_x^{c1}$ and $\mathbf{W}_x^{c2}$, we look at the influence of each when considering a single element from each recurrent state, $\mathbf{h}_t^{c1}$ and $\mathbf{h}_t^{c2}$:

$$\begin{aligned} h_{t,i}^{c1} &= g(\nu_i, \theta_i) h_{t-1,i}^{c1} - \phi(\nu_i, \theta_i) h_{t-1,i}^{c2} + \gamma_i \sum_{j=0}^d w_{x,(i,j)}^{c1} x_{t,j} \\ h_{t,i}^{c2} &= g(\nu_i, \theta_i) h_{t-1,i}^{c2} + \phi(\nu_i, \theta_i) h_{t-1,i}^{c1} + \gamma_i \sum_{j=0}^d w_{x,(i,j)}^{c2} x_{t,j}. \end{aligned} \quad (26)$$

We then get:

$$\begin{aligned} \frac{\partial h_{t,i}^{c1}}{\partial w_{x,(i,j)}^{c1}} &= g(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c1}}{\partial w_{x,(i,j)}^{c1}} - \phi(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c2}}{\partial w_{x,(i,j)}^{c1}} + \gamma_i x_{t,j} \\ \frac{\partial h_{t,i}^{c2}}{\partial w_{x,(i,j)}^{c1}} &= g(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c2}}{\partial w_{x,(i,j)}^{c1}} + \phi(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c1}}{\partial w_{x,(i,j)}^{c1}} \\ \frac{\partial h_{t,i}^{c1}}{\partial w_{x,(i,j)}^{c2}} &= g(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c1}}{\partial w_{x,(i,j)}^{c2}} - \phi(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c2}}{\partial w_{x,(i,j)}^{c2}} \\ \frac{\partial h_{t,i}^{c2}}{\partial w_{x,(i,j)}^{c2}} &= g(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c2}}{\partial w_{x,(i,j)}^{c2}} + \phi(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c1}}{\partial w_{x,(i,j)}^{c2}} + \gamma_i x_{t,j}. \end{aligned} \quad (27)$$

We see that each $h_{t,i}^{c1}$ gets affected by weights from only one row of $\mathbf{W}_x^{c1}$, thus, $\frac{\partial \mathbf{h}_t^{c1}}{\partial \mathbf{W}_x^{c1}}$ can be written as a matrix of the same dimension as $\mathbf{W}_x^{c1}$. The same is true for $\frac{\partial \mathbf{h}_t^{c2}}{\partial \mathbf{W}_x^{c2}}$, $\frac{\partial \mathbf{h}_t^{c1}}{\partial \mathbf{W}_x^{c2}}$, and $\frac{\partial \mathbf{h}_t^{c2}}{\partial \mathbf{W}_x^{c1}}$.

C.3 Single Layer Non-Linear RTUs Formulation:

We extend the linear RTUs to non-linear RTUs by adding a non-linear activation function $\mathbf{f}$ to the recurrent states. We can write the non-linear RTUs as follows:

$$\begin{aligned} \mathbf{h}_t^{c1} &= \mathbf{f}(\mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta}) \odot \mathbf{h}_{t-1}^{c1} - \boldsymbol{\phi}(\boldsymbol{\nu}, \boldsymbol{\theta}) \odot \mathbf{h}_{t-1}^{c2} + \boldsymbol{\gamma} \odot \mathbf{W}_x^{c1} \mathbf{x}_t) \\ \mathbf{h}_t^{c2} &= \mathbf{f}(\mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta}) \odot \mathbf{h}_{t-1}^{c2} + \boldsymbol{\phi}(\boldsymbol{\nu}, \boldsymbol{\theta}) \odot \mathbf{h}_{t-1}^{c1} + \boldsymbol{\gamma} \odot \mathbf{W}_x^{c2} \mathbf{x}_t) \\ \mathbf{h}_t &= [\mathbf{h}_t^{c1}; \mathbf{h}_t^{c2}] \end{aligned} \quad (28)$$

where $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$. Following the same procedure as in the linear case, we can derive RTRL update rules for the non-linear RTUs.

$$\begin{aligned} \frac{\partial \mathbf{h}_t^{c1}}{\partial \boldsymbol{\nu}^{\log}} &= \mathbf{f}'(\cdot) \left(\frac{\partial \mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta})}{\partial \boldsymbol{\nu}^{\log}} \odot \mathbf{h}_{t-1}^{c1} + \mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta}) \frac{\partial \mathbf{h}_{t-1}^{c1}}{\partial \boldsymbol{\nu}^{\log}} - \frac{\partial \boldsymbol{\phi}(\boldsymbol{\nu}, \boldsymbol{\theta})}{\partial \boldsymbol{\nu}^{\log}} \odot \mathbf{h}_{t-1}^{c2} - \boldsymbol{\phi}(\boldsymbol{\nu}, \boldsymbol{\theta}) \frac{\partial \mathbf{h}_{t-1}^{c2}}{\partial \boldsymbol{\nu}^{\log}} \right) \\ \frac{\partial \mathbf{h}_t^{c2}}{\partial \boldsymbol{\nu}^{\log}} &= \mathbf{f}'(\cdot) \left(\frac{\partial \mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta})}{\partial \boldsymbol{\nu}^{\log}} \odot \mathbf{h}_{t-1}^{c2} + \mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta}) \frac{\partial \mathbf{h}_{t-1}^{c2}}{\partial \boldsymbol{\nu}^{\log}} + \frac{\partial \boldsymbol{\phi}(\boldsymbol{\nu}, \boldsymbol{\theta})}{\partial \boldsymbol{\nu}^{\log}} \odot \mathbf{h}_{t-1}^{c1} + \boldsymbol{\phi}(\boldsymbol{\nu}, \boldsymbol{\theta}) \frac{\partial \mathbf{h}_{t-1}^{c1}}{\partial \boldsymbol{\nu}^{\log}} \right) \end{aligned} \quad (29)$$

$$\begin{aligned} \frac{\partial \mathbf{h}_t^{c1}}{\partial \boldsymbol{\theta}^{\log}} &= \mathbf{f}'(\cdot) \left(\frac{\partial \mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}^{\log}} \odot \mathbf{h}_{t-1}^{c1} + \mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta}) \frac{\partial \mathbf{h}_{t-1}^{c1}}{\partial \boldsymbol{\theta}^{\log}} - \frac{\partial \boldsymbol{\phi}(\boldsymbol{\nu}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}^{\log}} \odot \mathbf{h}_{t-1}^{c2} - \boldsymbol{\phi}(\boldsymbol{\nu}, \boldsymbol{\theta}) \frac{\partial \mathbf{h}_{t-1}^{c2}}{\partial \boldsymbol{\theta}^{\log}} \right) \\ \frac{\partial \mathbf{h}_t^{c2}}{\partial \boldsymbol{\theta}^{\log}} &= \mathbf{f}'(\cdot) \left(\frac{\partial \mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}^{\log}} \odot \mathbf{h}_{t-1}^{c2} + \mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta}) \frac{\partial \mathbf{h}_{t-1}^{c2}}{\partial \boldsymbol{\theta}^{\log}} + \frac{\partial \boldsymbol{\phi}(\boldsymbol{\nu}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}^{\log}} \odot \mathbf{h}_{t-1}^{c1} + \boldsymbol{\phi}(\boldsymbol{\nu}, \boldsymbol{\theta}) \frac{\partial \mathbf{h}_{t-1}^{c1}}{\partial \boldsymbol{\theta}^{\log}} \right) \end{aligned} \quad (30)$$

$$\begin{aligned} h_{t,i}^{c1} &= f(g(\nu_i, \theta_i) h_{t-1,i}^{c1} - \phi(\nu_i, \theta_i) h_{t-1,i}^{c2} + \gamma_i \sum_{j=0}^d w_{x,(i,j)}^{c1} x_{t,j}) \\ h_{t,i}^{c2} &= f(g(\nu_i, \theta_i) h_{t-1,i}^{c2} + \phi(\nu_i, \theta_i) h_{t-1,i}^{c1} + \gamma_i \sum_{j=0}^d w_{x,(i,j)}^{c2} x_{t,j}) \end{aligned} \quad (31)$$

We then get:

$$\begin{aligned}
\frac{\partial h_{t,i}^{c_1}}{\partial W_{x,(i,j)}^{c_1}} &= f'(\cdot)(g(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c_1}}{\partial W_{x,(i,j)}^{c_1}} - \phi(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c_2}}{\partial W_{x,(i,j)}^{c_1}} + \gamma_i x_{t,j}) \\
\frac{\partial h_{t,i}^{c_2}}{\partial W_{x,(i,j)}^{c_1}} &= f'(\cdot)(g(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c_2}}{\partial W_{x,(i,j)}^{c_1}} + \phi(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c_1}}{\partial W_{x,(i,j)}^{c_1}}) \\
\frac{\partial h_{t,i}^{c_1}}{\partial W_{x,(i,j)}^{c_2}} &= f'(\cdot)(g(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c_1}}{\partial W_{x,(i,j)}^{c_2}} - \phi(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c_2}}{\partial W_{x,(i,j)}^{c_2}}) \\
\frac{\partial h_{t,i}^{c_2}}{\partial W_{x,(i,j)}^{c_2}} &= f'(\cdot)(g(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c_2}}{\partial W_{x,(i,j)}^{c_2}} + \phi(\nu_i, \theta_i) \frac{\partial h_{t-1,i}^{c_1}}{\partial W_{x,(i,j)}^{c_2}} + \gamma_i x_{t,j})
\end{aligned} \tag{32}$$

C.4 Complexity Analysis of RTUs

We now move to calculate the computation and memory complexity of RTUs when learning using the RTRL rules introduced in the previous section.

For an input $\mathbf{x}_t \in \mathbb{R}^d$ and hidden states $\mathbf{h}_t = [\mathbf{f}(\mathbf{h}_t^{c_1}); \mathbf{f}(\mathbf{h}_t^{c_2})] \in \mathbb{R}^{2n}$, we have $\mathbf{g}(\boldsymbol{\nu}, \boldsymbol{\theta}), \phi(\boldsymbol{\nu}, \boldsymbol{\theta}), \boldsymbol{\gamma} \in \mathbb{R}^n$ and $\mathbf{W}_x^{c_1}, \mathbf{W}_x^{c_2} \in \mathbb{R}^{d \times n}$.

An agent using the RTU with RTRL needs to store the gradient information, $\frac{\partial \mathbf{h}_{t-1}^{c_1}}{\partial \boldsymbol{\psi}}$ and $\frac{\partial \mathbf{h}_{t-1}^{c_2}}{\partial \boldsymbol{\psi}}$, from one step to the next. We denote the set of saved gradient information as:

$$\begin{aligned}
\nabla_{\boldsymbol{\nu}^{t-1}} &\doteq \left\{ \frac{\partial \mathbf{h}_{t-1}^{c_1}}{\partial \boldsymbol{\nu}^{\log}}, \frac{\partial \mathbf{h}_{t-1}^{c_2}}{\partial \boldsymbol{\nu}^{\log}} \right\} \\
\nabla_{\boldsymbol{\theta}^{t-1}} &\doteq \left\{ \frac{\partial \mathbf{h}_{t-1}^{c_1}}{\partial \boldsymbol{\theta}^{\log}}, \frac{\partial \mathbf{h}_{t-1}^{c_2}}{\partial \boldsymbol{\theta}^{\log}} \right\} \\
\nabla_{\mathbf{W}_x^{t-1}} &\doteq \left\{ \frac{\partial \mathbf{h}_{t-1}^{c_1}}{\partial \mathbf{W}_x^{c_1}}, \frac{\partial \mathbf{h}_{t-1}^{c_2}}{\partial \mathbf{W}_x^{c_1}}, \frac{\partial \mathbf{h}_{t-1}^{c_1}}{\partial \mathbf{W}_x^{c_2}}, \frac{\partial \mathbf{h}_{t-1}^{c_2}}{\partial \mathbf{W}_x^{c_2}} \right\}.
\end{aligned} \tag{33}$$

The saved gradient information has the following dimensions:

$$\begin{aligned}
\nabla_{\boldsymbol{\nu}^{t-1}} &\in \mathbb{R}^{2n} \\
\nabla_{\boldsymbol{\theta}^{t-1}} &\in \mathbb{R}^{2n} \\
\nabla_{\mathbf{W}_x^{t-1}} &\in \mathbb{R}^{4(d \times n)}.
\end{aligned} \tag{34}$$

Then, it follows that memory complexity for RTU with RTRL is $\mathcal{O}(n + nd)$. i.e., linear in the number of parameters.

For the computational complexity, a forward pass according to 2 has a computational complexity of $\mathcal{O}(n + nd)$. Additionally, after doing the forward pass, the learner needs to update the saved gradient information according to equations 23 through 27 which has a computational complexity of $\mathcal{O}(n + nd)$. To summarize, using Real-Time Recurrent Learning with RTUs has linear computational and memory complexities.

C.5 Multi-Layers Recurrent Trace Units

We now extend RTUs to a multilayer setting. We show that in the multilayer case, we lose the computational advantages. However, prior work suggested that treating each recurrent layer independently is a sensible choice and allows us to gain a computational advantage[17]. Consider an RTU with n layers, we refer to the hidden dimension of a layer i where $0 < i \leq n$ as d_i . The network has the following set of parameters:

$$\begin{aligned}
\boldsymbol{\psi} &\doteq \{\boldsymbol{\psi}_1, \boldsymbol{\psi}_2, \boldsymbol{\psi}_3, \dots, \boldsymbol{\psi}_n\}, \\
\boldsymbol{\psi}_i &\doteq \{\boldsymbol{\nu}^{\log, i}, \boldsymbol{\theta}^{\log, i}, \mathbf{W}_x^{c_1, i}, \mathbf{W}_x^{c_2, i}\}
\end{aligned} \tag{35}$$

To update the network parameters, we need to calculate the gradient of the loss w.r.t the parameters from all the layers:

$$\begin{aligned}\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{\psi}} &= \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t^n} \frac{\partial \mathbf{h}_t^n}{\partial \boldsymbol{\psi}} \\ \frac{\partial \mathbf{h}_t^n}{\partial \boldsymbol{\psi}} &= \left\{ \frac{\partial \mathbf{h}_t^n}{\partial \boldsymbol{\psi}_1}, \frac{\partial \mathbf{h}_t^n}{\partial \boldsymbol{\psi}_2}, \frac{\partial \mathbf{h}_t^n}{\partial \boldsymbol{\psi}_3}, \dots, \frac{\partial \mathbf{h}_t^n}{\partial \boldsymbol{\psi}_n} \right\}\end{aligned}\quad (36)$$

where

$$\mathbf{h}_t^n = \begin{bmatrix} \mathbf{f}(\mathbf{h}_t^{c1,n}) \\ \mathbf{f}(\mathbf{h}_t^{c2,n}) \end{bmatrix} \quad (37)$$

Take as an example the gradient of $\mathbf{h}_t^{c1,n}$ w.r.t $\boldsymbol{\nu}^{\log,n}, \boldsymbol{\nu}^{\log,n-1}, \dots, \boldsymbol{\nu}^{\log,1}$. Unrolling the last two layers of the network, we get:

$$\begin{aligned}\mathbf{h}_t^{c1,n} &= \mathbf{g}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \odot \mathbf{h}_{t-1}^{c1,n} - \boldsymbol{\phi}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \odot \mathbf{h}_{t-1}^{c2,n} + \boldsymbol{\gamma}^n \odot \mathbf{W}_x^{c1,n} \mathbf{h}_t^{n-1} \\ &= \mathbf{g}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \odot \mathbf{h}_{t-1}^{c1,n} - \boldsymbol{\phi}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \odot \mathbf{h}_{t-1}^{c2,n} + \boldsymbol{\gamma}^n \odot \mathbf{W}_x^{c1,n} \begin{bmatrix} \mathbf{f}(\mathbf{h}_t^{c1,n-1}) \\ \mathbf{f}(\mathbf{h}_t^{c2,n-1}) \end{bmatrix} \\ &= \mathbf{g}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \odot \mathbf{h}_{t-1}^{c1,n} - \boldsymbol{\phi}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \odot \mathbf{h}_{t-1}^{c2,n} + \boldsymbol{\gamma}^n \odot \mathbf{W}_x^{c1,n} \\ &\quad \left[\mathbf{f}(\mathbf{g}(\boldsymbol{\nu}^{n-1}, \boldsymbol{\theta}^{n-1}) \odot \mathbf{h}_{t-1}^{c1,n-1} - \boldsymbol{\phi}(\boldsymbol{\nu}^{n-1}, \boldsymbol{\theta}^{n-1}) \odot \mathbf{h}_{t-1}^{c2,n-1} + \boldsymbol{\gamma}^{n-1} \odot \mathbf{W}_x^{c1,n-1} \mathbf{h}_t^{n-2}) \right. \\ &\quad \left. \mathbf{f}(\mathbf{g}(\boldsymbol{\nu}^{n-1}, \boldsymbol{\theta}^{n-1}) \odot \mathbf{h}_{t-1}^{c2,n-1} + \boldsymbol{\phi}(\boldsymbol{\nu}^{n-1}, \boldsymbol{\theta}^{n-1}) \odot \mathbf{h}_{t-1}^{c1,n-1} + \boldsymbol{\gamma}^{n-1} \odot \mathbf{W}_x^{c2,n-1} \mathbf{h}_t^{n-2}) \right]\end{aligned}\quad (38)$$

The gradient of $\mathbf{h}_t^{c1,n}$ w.r.t $\boldsymbol{\nu}^{\log,n}$ can be calculated in linear complexity as indicated in the previous section.

Calculating the gradient w.r.t the parameters from the earlier layer:

$$\begin{aligned}\frac{\partial \mathbf{h}_t^{c1,n}}{\partial \boldsymbol{\nu}^{\log,n-1}} &= \mathbf{g}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \frac{\partial \mathbf{h}_{t-1}^{c1,n}}{\partial \boldsymbol{\nu}^{\log,n-1}} - \boldsymbol{\phi}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \frac{\partial \mathbf{h}_{t-1}^{c2,n}}{\partial \boldsymbol{\nu}^{\log,n-1}} + \boldsymbol{\gamma}^n \odot \mathbf{W}_x^{c1,n} \frac{\partial \mathbf{h}_t^{n-1}}{\partial \boldsymbol{\nu}^{\log,n-1}} \\ \frac{\partial \mathbf{h}_t^{n-1}}{\partial \boldsymbol{\nu}^{\log,n-1}} &\in \mathbf{R}^{2d_{n-1}} \text{ Can be calculated with linear complexity.} \\ \frac{\partial \mathbf{h}_{t-1}^{c1,n}}{\partial \boldsymbol{\nu}^{\log,n-1}} &\in \mathbf{R}^{d_n \times d_{n-1}} \text{ Saved from previous timestep.} \\ \frac{\partial \mathbf{h}_{t-1}^{c2,n}}{\partial \boldsymbol{\nu}^{\log,n-1}} &\in \mathbf{R}^{d_n \times d_{n-1}} \text{ Saved from previous timestep.}\end{aligned}\quad (39)$$

$$\begin{aligned}\frac{\partial \mathbf{h}_t^{c1,n}}{\partial \boldsymbol{\nu}^{\log,n-2}} &= \mathbf{g}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \frac{\partial \mathbf{h}_{t-1}^{c1,n}}{\partial \boldsymbol{\nu}^{\log,n-2}} - \boldsymbol{\phi}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \frac{\partial \mathbf{h}_{t-1}^{c2,n}}{\partial \boldsymbol{\nu}^{\log,n-2}} + \boldsymbol{\gamma}^n \odot \mathbf{W}_x^{c1,n} \frac{\partial \mathbf{h}_t^{n-1}}{\partial \boldsymbol{\nu}^{\log,n-2}} \\ &= \mathbf{g}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \frac{\partial \mathbf{h}_{t-1}^{c1,n}}{\partial \boldsymbol{\nu}^{\log,n-2}} - \boldsymbol{\phi}(\boldsymbol{\nu}^n, \boldsymbol{\theta}^n) \frac{\partial \mathbf{h}_{t-1}^{c2,n}}{\partial \boldsymbol{\nu}^{\log,n-2}} + \boldsymbol{\gamma}^n \odot \mathbf{W}_x^{c1,n} \\ &\quad \mathbf{f}'(\cdot) \left[\mathbf{g}(\boldsymbol{\nu}^{n-1}, \boldsymbol{\theta}^{n-1}) \frac{\partial \mathbf{h}_{t-1}^{c1,n-1}}{\partial \boldsymbol{\nu}^{\log,n-2}} - \boldsymbol{\phi}(\boldsymbol{\nu}^{n-1}, \boldsymbol{\theta}^{n-1}) \frac{\partial \mathbf{h}_{t-1}^{c2,n-1}}{\partial \boldsymbol{\nu}^{\log,n-2}} + \boldsymbol{\gamma}^{n-1} \odot \mathbf{W}_x^{c1,n-1} \frac{\partial \mathbf{h}_t^{n-2}}{\partial \boldsymbol{\nu}^{\log,n-2}} \right. \\ &\quad \left. \mathbf{g}(\boldsymbol{\nu}^{n-1}, \boldsymbol{\theta}^{n-1}) \frac{\partial \mathbf{h}_{t-1}^{c2,n-1}}{\partial \boldsymbol{\nu}^{\log,n-2}} + \boldsymbol{\phi}(\boldsymbol{\nu}^{n-1}, \boldsymbol{\theta}^{n-1}) \frac{\partial \mathbf{h}_{t-1}^{c1,n-1}}{\partial \boldsymbol{\nu}^{\log,n-2}} + \boldsymbol{\gamma}^{n-1} \odot \mathbf{W}_x^{c2,n-1} \frac{\partial \mathbf{h}_t^{n-2}}{\partial \boldsymbol{\nu}^{\log,n-2}} \right] \\ \frac{\partial \mathbf{h}_t^{n-2}}{\partial \boldsymbol{\nu}^{\log,n-2}} &\in \mathbf{R}^{2d_{n-2}} \text{ Can be calculated with linear complexity.} \\ \frac{\partial \mathbf{h}_{t-1}^{c1,n-1}}{\partial \boldsymbol{\nu}^{\log,n-2}} &\in \mathbf{R}^{d_{n-1} \times d_{n-2}} \text{ Saved from previous timestep.} \\ \frac{\partial \mathbf{h}_{t-1}^{c2,n-1}}{\partial \boldsymbol{\nu}^{\log,n-2}} &\in \mathbf{R}^{d_{n-1} \times d_{n-2}} \text{ Saved from previous timestep.} \\ \frac{\partial \mathbf{h}_{t-1}^{c1,n}}{\partial \boldsymbol{\nu}^{\log,n-2}} &\in \mathbf{R}^{d_n \times d_{n-2}} \text{ Saved from previous timestep.} \\ \frac{\partial \mathbf{h}_{t-1}^{c2,n}}{\partial \boldsymbol{\nu}^{\log,n-2}} &\in \mathbf{R}^{d_n \times d_{n-2}} \text{ Saved from previous timestep.}\end{aligned}\quad (40)$$

Let's define $J_{i,i-1} = \left\{ \frac{\partial \mathbf{h}_{t-1}^{c1,i}}{\partial \mathbf{v}_{\log,i-1}^{c1,i}}, \frac{\partial \mathbf{h}_{t-1}^{c2,i}}{\partial \mathbf{v}_{\log,i-1}^{c2,i}} \right\}$. Then, to calculate the gradient of the hidden units from layer n w.r.t the parameters from layer $n-1$, we need save $J_{n,n-1}$, and to calculate the gradient of the hidden units from layer n w.r.t the parameters of layer $n-2$, we need to save $J_{n,n-2}$ and $J_{n-1,n-2}$. If we keep going, to calculate the gradient of the hidden units of layer n w.r.t the parameters of the first layer, we need to save $J_{n,1}, J_{n-1,1}, J_{n-2,1}, \dots, J_{2,1}$.

C.6 Implementing RTRL within the reverse-mode automatic differentiation

For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, we have the Jacobian $\partial f(x) \in \mathbb{R}^{m \times n}$ and we can calculate this Jacobian in two ways: forward-mode or reverse-mode differentiation. In the forward-mode differentiation, the chain rule is applied to each operation while traversing the computational graph in the forward pass [2]. While computing the derivatives during the forward pass is appealing, we need to do n forward passes to get the full Jacobian, where each forward pass would allow us to compute the derivative w.r.t only one of the inputs. i.e., with forward mode differentiation, we evaluate the Jacobian one column at a time. As a result, forward-mode differentiation is inefficient for neural networks; neural networks map from learnable parameters, which can be in millions, to a loss function, hence, have very wide jacobians, $n \gg m$.

Reverse-mode differentiation, on the other hand, offers a more efficient approach. It allows us to evaluate the Jacobian one row at a time, which is particularly advantageous for neural networks. This efficiency comes at the cost of two passes through the network: a forward pass for function evaluation and a backward pass for derivative evaluation [2].

RTRL is an instance of forward-mode differentiation; during the forward pass, the gradient information is evaluated along with the recurrent function computation. As a result, there is no need to perform a backward pass for the recurrent component. To efficiently use a recurrent layer with RTRL within a larger neural network, we combine RTRL for the recurrent layer with the reverse mode for the rest of the network. We use a stop gradient operation on the recurrent layer hidden state and do a normal reverse-mode differentiation. Due to the stop gradient operation, the gradient from the reverse mode assumes no time dependencies between the recurrent states. We then use the gradient traces calculated during the forward pass of the recurrent layer to correct the gradient from the reverse mode and account for the time dependencies between the recurrent states [3].³

D Issues with Two Alternative Parameterizations

In this section we provide a few additional insights on alternative ways to handle complex numbers within an RNN, and why they are not preferable.

D.1 Stability

We look at the gradient when using each complex representation to understand how different representations affect learning stability. Since each hidden unit has only one recurrent relation in diagonal RNNs, it is sufficient to consider one unit, in isolation. To keep the below intuition simple, we also omit the input of $\mathbf{x}$, and consider

$$h_t = \lambda h_{t-1} = \dots = \lambda^t h_0 \quad (41)$$

where h_0 is the initial hidden state.

Real Representation $a + bi$: Substituting λ in (41) with the real representation, we get:

$$h_t = (a + bi)^t h_0 = h_0 \sum_{k=0}^t \binom{t}{k} a^{t-k} b^k i^k$$

³This can be implemented by defining a custom vjp for the recurrent layer, which modifies the backward pass for the recurrent layer to include the gradient traces.

Then it follows that the gradient w.r.t the learnable parameters a and b is:

$$\begin{aligned}\frac{\partial h_t}{\partial a} &= h_0 \sum_{k=0}^t \binom{t}{k} (t-k) a^{t-k-1} b^k i^k \\ \frac{\partial h_t}{\partial b} &= h_0 \sum_{k=0}^t \binom{t}{k} k a^{t-k} b^{k-1} i^k\end{aligned}$$

To prevent the gradient from vanishing/exploding, we need to restrict both $|a|$ and $|b|$ to be $\in (0, 1]$.

Exponential Representation $r \exp(i\theta)$: Substituting λ in (41) with the exponential representation, we get:

$$h_t = r^t \exp(it\theta) h_0$$

and the gradient w.r.t the learnable parameters r and θ is:

$$\frac{\partial h_t}{\partial r} = t r^{t-1} \exp(it\theta) h_0, \quad \frac{\partial h_t}{\partial \theta} = r^t \exp(it\theta) i t h_0$$

To prevent the gradient from vanishing/exploding, we need to restrict $r \in (0, 1]$.

Cosine Representation $r(\cos(\theta) + i \sin(\theta))$: Substituting λ in (41) with the cosine representation, we get:

$$h_t = r^t (\cos(t\theta) + i \sin(t\theta)) h_0$$

and the gradient w.r.t the learnable parameters r and θ is:

$$\begin{aligned}\frac{\partial h_t}{\partial r} &= t r^{t-1} (\cos(t\theta) + i \sin(t\theta)) h_0 \\ \frac{\partial h_t}{\partial \theta} &= r^t (i t \cos(t\theta) - t \sin(t\theta)) h_0.\end{aligned}$$

To prevent the gradient from vanishing/exploding, we need to restrict $r \in (0, 1]$. It is simpler to maintain stability with the exponential and cosine representations, since we only need to constrain $r \in (0, 1]$, whereas the real representation requires us to restrict both the complex number's magnitude and phase.

D.2 Biased gradient when only using the real part of the hidden state

We can attempt to get the benefits of having a real-valued hidden state by simply converting the complex-valued state to a real-valued one within the LRU. However, taking only the real part results in a biased gradient, as we show in this section.

Consider again the one recurrent unit example, but now also consider the output obtained by taking only the real part of the recurrent state: $y_t = w \operatorname{Re}\{h_t\}$, where w is a learnable parameter. The gradient w.r.t r and θ is:

$$\begin{aligned}\frac{\partial y_t}{\partial r} &= w \left(\cos(\theta) h_{t-1} + r \cos(\theta) \frac{\partial h_{t-1}}{\partial r} \right) \\ \frac{\partial y_t}{\partial \theta} &= w \left(-r \sin(\theta) h_{t-1} + r \cos(\theta) \frac{\partial h_{t-1}}{\partial \theta} \right)\end{aligned}\tag{42}$$

Multiplying by a complex number z is equivalent to a rotation by the matrix $\begin{bmatrix} \operatorname{Re}\{z\} & -\operatorname{Im}\{z\} \\ \operatorname{Im}\{z\} & \operatorname{Re}\{z\} \end{bmatrix}$

and a scale by $\sqrt{\operatorname{Re}\{z\}^2 + \operatorname{Im}\{z\}^2}$. We re-write the recurrent unit using this property as:

$$\begin{aligned}h_t^{c1} &= r \cos(\theta) h_{t-1}^{c1} - r \sin(\theta) h_{t-1}^{c2} \\ h_t^{c2} &= r \cos(\theta) h_{t-1}^{c2} + r \sin(\theta) h_{t-1}^{c1} \\ y_t &= w (h_t^{c1} + h_t^{c2})\end{aligned}\tag{43}$$

Notice that we don't need to take the real part of the recurrent state in this formulation. We now write the gradient using this new formulation:

$$\begin{aligned}
\frac{\partial y_t}{\partial r} &= w(r \cos(\theta)(h_{t-1}^{c_1} + h_{t-1}^{c_2}) + r \cos(\theta)(\frac{\partial h_{t-1}^{c_1}}{\partial r} + \frac{\partial h_{t-1}^{c_2}}{\partial r})) \\
&\quad + r \sin(\theta)(h_{t-1}^{c_1} - h_{t-1}^{c_2}) + r \sin(\theta)(\frac{\partial h_{t-1}^{c_1}}{\partial r} - \frac{\partial h_{t-1}^{c_2}}{\partial r})) \\
\frac{\partial y_t}{\partial \theta} &= w(-r \sin(\theta)(h_{t-1}^{c_1} + h_{t-1}^{c_2}) + r \cos(\theta)(\frac{\partial h_{t-1}^{c_1}}{\partial \theta} + \frac{\partial h_{t-1}^{c_2}}{\partial \theta})) \\
&\quad + r \cos(\theta)(h_{t-1}^{c_1} - h_{t-1}^{c_2}) + r \sin(\theta)(\frac{\partial h_{t-1}^{c_1}}{\partial \theta} - \frac{\partial h_{t-1}^{c_2}}{\partial \theta}))
\end{aligned} \tag{44}$$

Comparing Eq. 44 and Eq. 42, we can see that using only the real part of the recurrent state leads to a loss of information in the gradient.

E Trace Conditioning Experiments

In animal learning, *Trace conditioning* is a type of experiment where animals predict the occurrence of a stimulus (e.g., food), based on the occurrence of another stimulus like a tone. There is no prior connection between the two stimuli. However, after enough repetitions of pairing them together—playing the tone and then serving the food—the animal learns to anticipate food arrival when it hears the tone [35]. We use an open-source *trace conditioning* benchmark introduced in prior work [38]. Two signals appear sequentially: the Conditional Stimulus (CS) and the Unconditional Stimulus (US). The CS is the trigger signal, similar to the tone, and the US is the signal of interest and appears several time steps after the CS, similar to the food. The agent also observes several distractor signals which are uncorrelated with the CS and US; the agent must learn ignore them and focus only on predicting the US.

The agent's objective is to predict the onset of the US, which we model as a prediction of the discounted sum of the future US, G_t :

$$G_t \doteq \sum_{k=0}^{\infty} \gamma^k \text{US}_{t+k+1}, \tag{45}$$

where γ is a discount factor determining the prediction horizon. This problem is challenging because the CS appears, then disappears, and sometime later the US appears; the agent must construct an internal state that represents the time period between the two signals.

E.1 Additional Trace Conditioning Experiments

Comparison to Other RTRL-based Architectures: We now compare RTUs to other RTRL-based approaches with similar architectures: an online version of LRU [47] and a vanilla block diagonal RNN. The block diagonal is a recurrent formulation similar to RTU but ignores the relation between the learnable parameters. i.e., replaces $\mathbf{c}_k$ in 3.2 with $\mathbf{c}_k = \begin{bmatrix} a_k & b_k \\ c_k & d_k \end{bmatrix}$.

The results in figure E.1 indicate that these seemingly small differences between the diagonal RNNs can result in significantly different behavior. RTUs outperform online LRUs, with the differences discussed in-depth in Section 3.4. RTUs also outperform the block diagonal RNN. We emphasized using real-valued diagonals implicitly assumes symmetric matrices, but that is for a single real-value. This block diagonal has more representational capacity than the RTU. This result suggests it is beneficial for learning to enforce these constraints on the learnable parameters, that they correspond to the rotational representation of complex numbers.

On the role of RTRL in RTUs: We investigated the importance of using RTRL rather than T-BPTT for RTUs. We found that RTUs with T-BPTT perform similarly to LRUs with T-BPTT. With a sufficiently large truncation, performance eventually matched that of RTRL, but with significantly more computation. See Appendix E.1 for the results.

To highlight the role of RTRL in RTUs, we evaluated the performance of both linear and non-linear RTUs with T-BPTT. In this experiment, we all agents use the same number of parameters; the only difference is whether they use RTRL or BPTT.

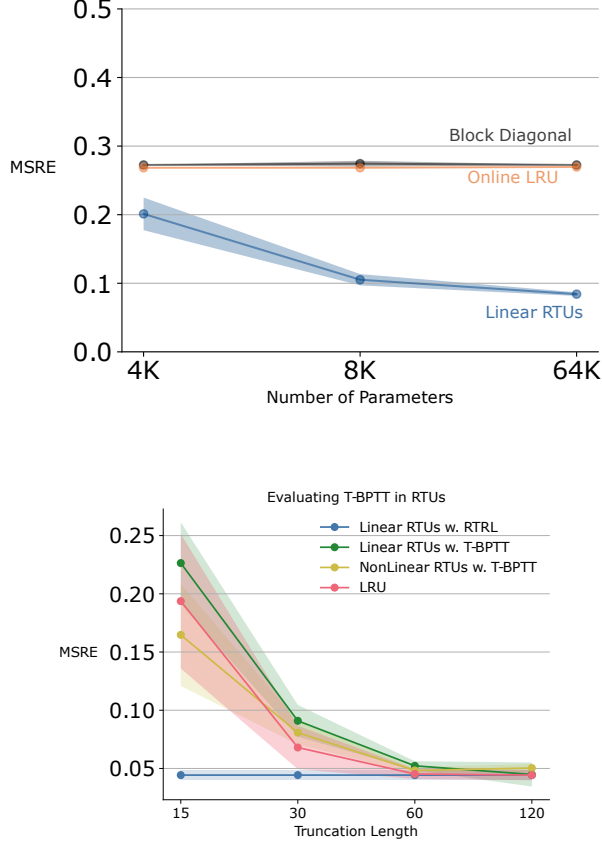


Figure 11: Evaluating RTUs with BPTT.

Figure E.1 summarises the results of this experiment. We can see that the performance of T-BPTT approaches the performance of RTRL as the truncation length increases to cover the whole context of the task.

E.2 Trace Conditioning Experiments Details

All agents have one recurrent layer, either an RTU or a GRU, and one linear layer. At each time step t , the agent passes the observation $\mathbf{o}_t$ to the recurrent layer, which outputs the recurrent state, the agent state. The recurrent state is then passed to the linear layer generating the prediction. For each agent, we swept over the learning rate α used to update the network parameters, $\alpha \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}\}$, and averaged the performance for each learning rate over 5 independent runs. We then selected the best-performing learning rate for each agent and ran 30 independent runs using it. For all the experiments, we ran the agents for 2 million steps, and the performance was the mean squared prediction error averaged over the 2 million steps.

E.3 Learning Rate Sensitivity

We show the learning rate sensitivity for all agents in the animal learning benchmark.

F Integrating Linear RTRL Methods with PPO

When performing batch updates, as with PPO, the RTRL gradients used to update the recurrent parameters will be stale, as they were calculated during the interaction with the environment w.r.t old policy and value parameters. One solution to mitigate the gradient staleness is to go through the whole trajectory after each epoch update and re-compute the gradient traces. However, this can be

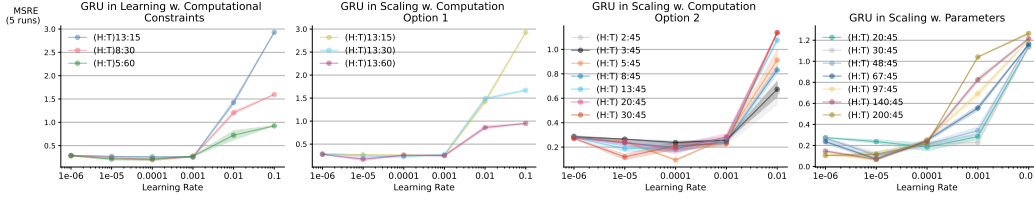


Figure 12: GRUs used in the animal learning benchmark. The $(H: T)$ in the label refers to the (hidden dimension: truncation length) for the GRU.

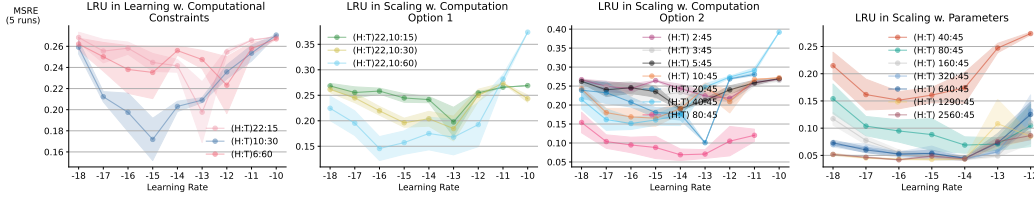


Figure 13: LRUs used in the animal learning benchmark. The $(H: T)$ in the label refers to the (hidden dimension: truncation length) for the GRU.

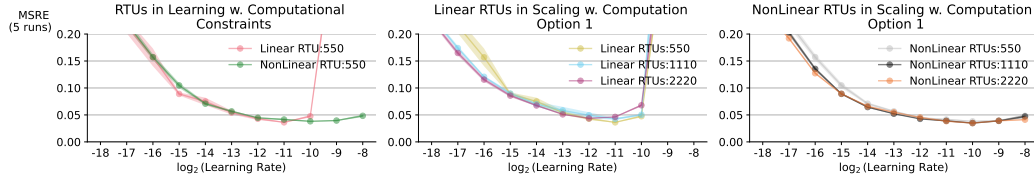


Figure 14: RTUs used in the animal learning benchmark.

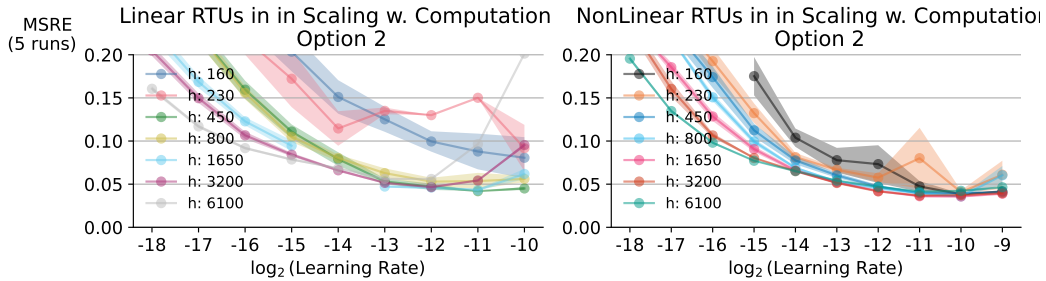


Figure 15: RTUs used in the animal learning benchmark.

computationally expensive. Algorithm 1, we provide the pseudocode for integrating RTRL methods with PPO with optional steps for re-running the network to update the RTRL gradient traces, the value targets, and the advantage estimates.

In the next experiment, we investigate the effect of the gradient staleness in RTRL when combined with PPO and how this staleness interacts with the targets and advantage estimates. To understand this interaction, we evaluate all combinations of stale gradient and stale targets with increasing the number of epoch updates. We perform this analysis on the Ant-P environment from the Mujoco POMDP benchmark [30, 11, 26, 31]. Surprisingly, Figure 17 shows that using a stale gradient results in better

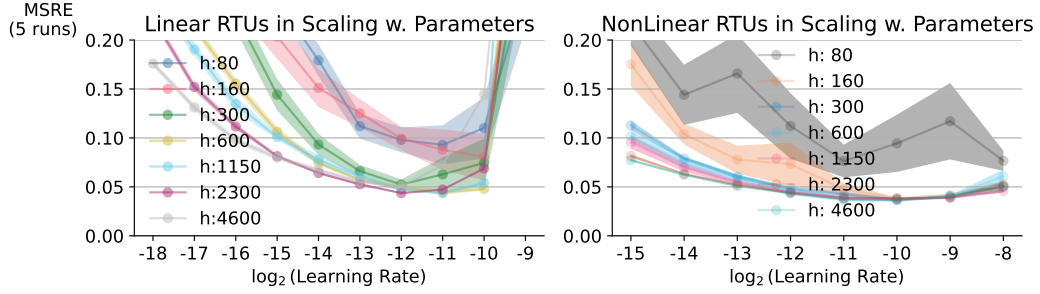


Figure 16: RTUs used in the animal learning benchmark.

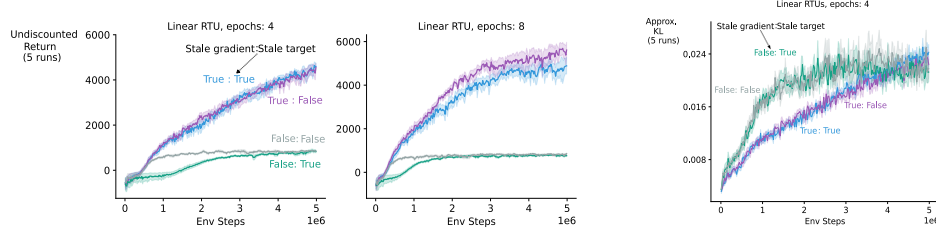


Figure 17: The impact of stale gradients and stale targets when combining RTRL and PPO on Ant.

performance with RTUs than re-computing the gradient traces. This performance improvement is also consistent when we increase the number of epochs from 4 to 8. It also shows that re-computing the value targets and advantage estimates has a minimal effect on the performance. We repeated the same experiments for NonLinear RTUs and Online LRU in Appendix F with consistent results.

One hypothesis for the superior performance of stale gradients is that the staleness is helping PPO maintain the trust region. We investigate this hypothesis by measuring the KL divergence between the policy used to collect the trajectory and the most recent policy. We use the following estimate for the KL divergence between the two policies as $(r - 1) - \log(r)$, where $r = (\pi_{\theta_{new}})/(\pi_{\theta_{old}})$. The rightmost subplot of Figure 17 shows that at the beginning of learning, agents with stale gradients move away from the old policy more slowly than agents with fresh gradients; perhaps stale gradients help maintaining the trust region.

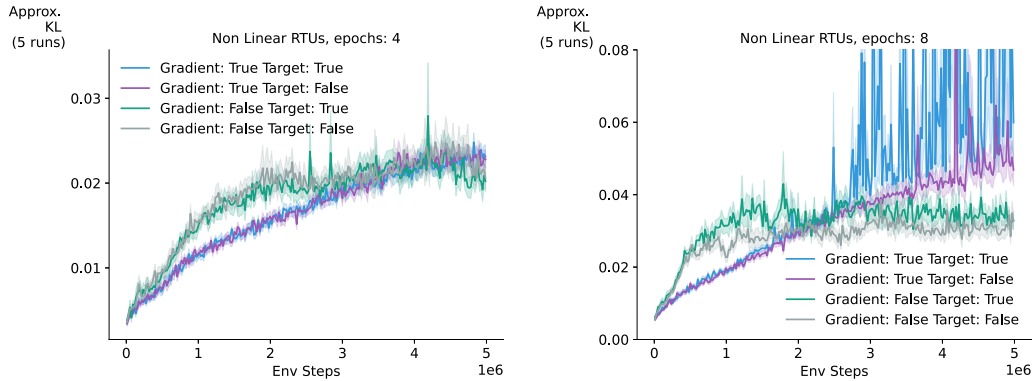


Figure 18: (a) Approximate KL divergence for NonLinear RTU with 4 epochs. (b) Approximate KL divergence for NonLinear RTU with 8 epochs.

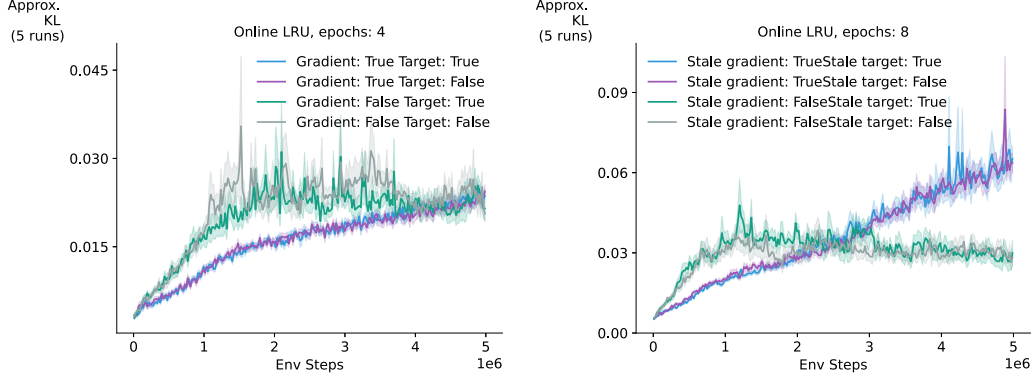


Figure 19: (a) Approximate KL divergence for LRU with 4 epochs.(b) Approximate KL divergence for LRU with 8 epochs.

Algorithm 1 Pseudocode for integrating RTRL methods with PPO

Inputs: a differentiable policy parametrization $\pi(a|s, \mathbf{W}_p)$.
Inputs: a differentiable state-value function parametrization $\hat{v}(s, \mathbf{W}_v)$.
loop
 Generate a trajectory using the current policy $\mathbf{O}_0, A_0, R_1, \dots, \mathbf{O}_M, A_M, R_M$,
 Store the transition and the gradient traces for the recurrent components along the way for $i = 0, \dots, M$
 Compute the advantage estimates and the target value for each timestep t
 for epoch = 1, ..., k **do**
 Divide the trajectory into minibatches and shuffle them.
 for minibatch = 1, ..., m **do**
 Calculate PPO loss
 Perform a gradient step with AutoDiff and correct it with the RTRL saved gradient as discussed in C.6.
 end for
 [Optional] Re-run network to update hidden states and the gradient traces for the trajectory.
 [Optional] Update value targets and advantages estimates for the trajectory.
 end for
end loop

G Memory-Based Control Experiments

In our implementations, we use a shared representation learning network followed by two MLPs for the actor and the critic heads, as illustrated in Figure 20. The shared representations consist of a feedforward layer with 64 hidden units and memory components: an RTU, LRU, or a GRU. The actor and the critic’s heads consist of two feedforward layers with tanh activation function.

Additional Mujoco Results:

In Figures 21 and 22, we set the truncation length for GRU and LRU to be 64, which is larger than needed to solve the Mujoco POMDP tasks. These results show that even when the truncation length is larger than needed, RTUs still outperform T-BPTT baselines. We also show that the transformer-based models, GPT2, perform worse than RNN-based models. This is consistent with previous work suggesting that transformers might not be suitable for RL tasks [31].

Learning Rate Sensitivity:

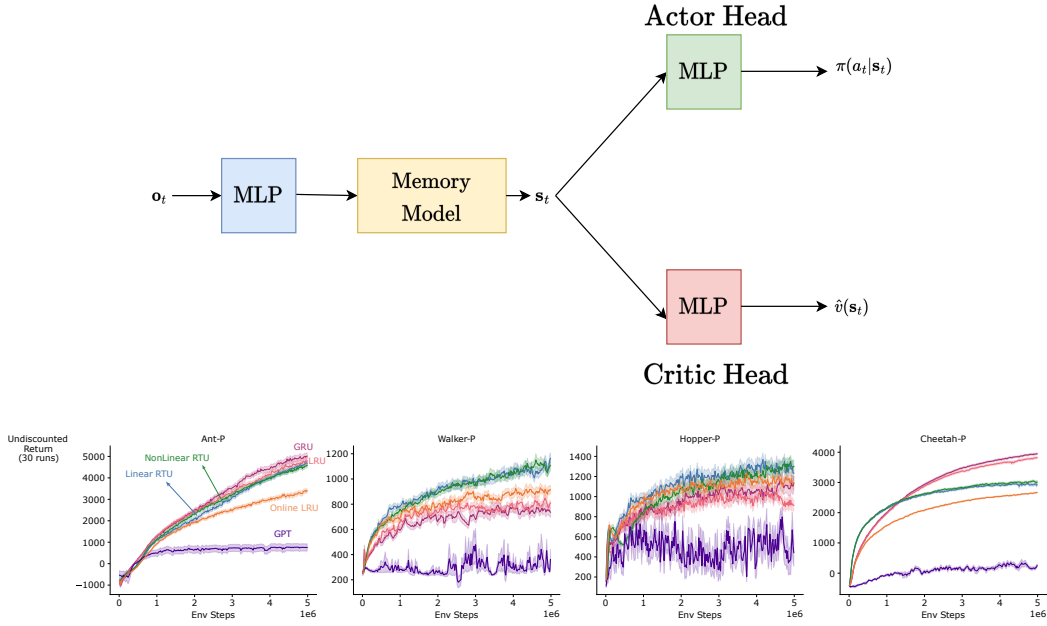


Figure 21: Additional results on Mujoco-P, where we allow GRU and LRU to use a larger truncation length than needed to solve these tasks. We also show results for GPT?

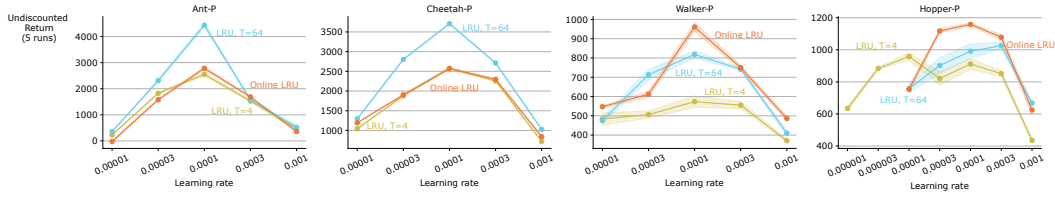


Figure 23: Learning rate sweep for LRU in the control experiments.

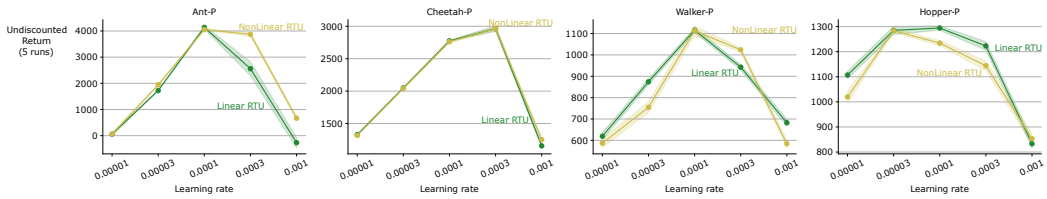


Figure 24: Learning rate sweep for RTUs in the control experiments.

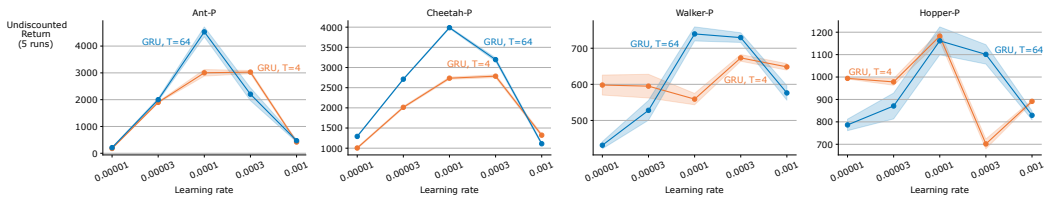


Figure 25: Learning rate sweep for GRUs in the control experiments.

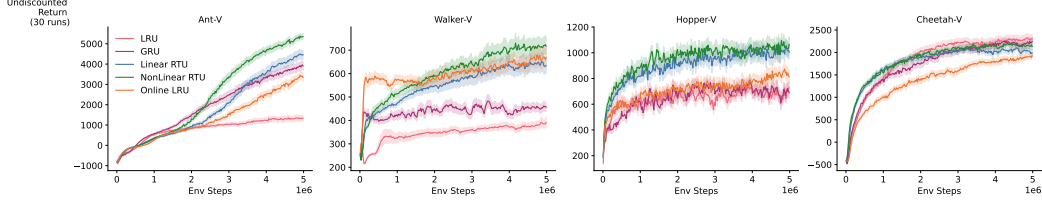


Figure 22: Additional results on Mujoco-P, where we allow GRU and LRU to use a larger truncation

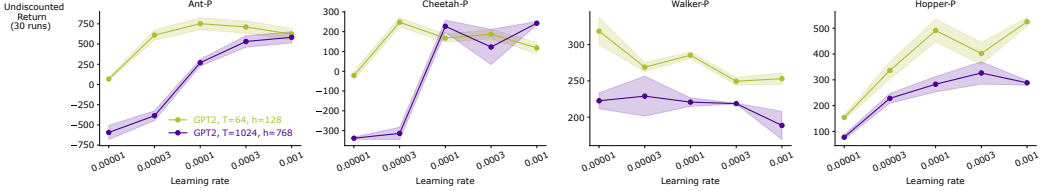


Figure 26: Learning rate sweep for GPT in the control experiments.

Figures 23, 24, 25, and 26 show the learning rate sensitivity for all agents in the Mujoco POMDP benchmark.

Finally, we used the default hyper-parameters for PPO [39] indicated in Table 1 for all agents.

Name	Value
Buffer size	2048
Num epochs	10
Number of Mini-batches	32
GAE, λ	0.95
Discount factor, γ	0.99
policy clip parameter	0.2
Value loss clip parameter	0.5
Gradient clip parameter	0.5
Optimizer	Adam
Optimizer step size	$[1e-05, 3e-05, 1e-04, 3e-04, 1e-03]$

Table 1: Hyper Parameters for PPO.

H Compute resources

We ran the Mujoco-P, Mujoco-V on NVIDIA P100 GPU. Each of the Mujoco-P and Mujoco-V trials took around 30 minutes to complete on a single GPU. For the POPGym experiments and animal learning experiments, we used a large CPU cluster. Each trial of the POPGym experiments took around 2 hours to complete. While each run of animal learning took around 15 minutes to complete on a single CPU with memory less than 1 GB.